

Felpe Luis Korbes, Celso Tadaki Sinoka, João Baréa e Silva

**Implementação de Segurança Integrada em
Redes LoRaWAN: Detecção de Anomalias via
TinyML e Criptografia Leve Ascon.**

São Paulo, SP

2026

Felpe Luis Korbes, Celso Tadaki Sinoka, João Baréa e Silva

Implementação de Segurança Integrada em Redes LoRaWAN: Detecção de Anomalias via TinyML e Criptografia Leve Ascon.

Trabalho de conclusão de curso apresentado
ao Departamento de Engenharia de Computa-
ção e Sistemas Digitais da Escola Politécnica
da Universidade de São Paulo para obtenção
do Título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Orientador: Prof. Dr. Victor Takashi Hayashi

São Paulo, SP

2026

Gerar a ficha catalográfica em <https://www.poli.usp.br/bibliotecas/servicos/catalogacao-na-publicacao>
Salvar o pdf e incluir na monografia

*Este trabalho é dedicado às crianças adultas que,
quando pequenas, sonharam em se tornar cientistas.*

Resumo

A segurança da informação no protocolo *Long Range Wide Area Network* LoRaWAN é um desafio crítico para aplicações de Internet das Coisas (IoT), devido à vulnerabilidade a ataques, como *jamming*, *spoofing*, DoS e *Man-in-the-middle*, combinada às rigorosas restrições energéticas e computacionais dos dispositivos de borda. Diante desse cenário, o presente trabalho tem como objetivo desenvolver e avaliar uma arquitetura de segurança integrada para dispositivos embarcados. A metodologia proposta baseia-se na implementação do algoritmo de criptografia leve Ascon, visando garantir a confidencialidade e integridade dos dados, operando em conjunto com um modelo de aprendizado de máquina focado na detecção de anomalias e intrusões no tráfego da rede. O protótipo será construído utilizando microcontroladores com restrição de recursos. Espera-se que a integração do padrão Ascon com a detecção com o aprendizado de máquina mitigue vetores de ataque comuns em redes LoRaWAN sem inviabilizar a operação sustentável dos dispositivos, resultando em uma solução robusta, autônoma e aplicável a ambientes de automação industrial e cidades inteligentes.

Palavras-chave: LoRaWAN; IoT; Detecção de Anomalias; Criptografia Leve.

Abstract

Information security in the LoRaWAN protocol is a critical challenge for Internet of Things (IoT) applications, due to vulnerability to attacks such as *jamming*, *spoofing*, DoS, and *Man-in-the-middle*, combined with the strict energy and computational constraints of edge devices. In this context, the present work aims to develop and evaluate an integrated security architecture for embedded devices. The proposed methodology is based on the implementation of the lightweight cryptographic algorithm Ascon to ensure data confidentiality and integrity, operating alongside a machine learning model focused on the detection of anomalies and intrusions in network traffic. The prototype will be built using resource-constrained microcontrollers. It is expected that integrating the Ascon standard with anomaly detection through machine learning will mitigate common attack vectors in LoRaWAN networks without compromising the sustainable operation of the devices, resulting in a robust, autonomous solution applicable to industrial automation and smart city environments.

Keywords: LoRaWAN; IoT; Anomaly Detection; Lightweight Cryptography.

Lista de ilustrações

Figura 1 – Histórico de versionamento da especificação da LoRaWAN	26
Figura 2 – Arquitetura de rede LoRaWAN	27
Figura 3 – (a) OTAA processo de entrada LoRaWAN 1.0.x. (b) OTAA processo de entrada LoRaWAN 1.1	29
Figura 4 – Classe para comunicacoes ED	30
Figura 5 – Estrutura e Criptografia das mensagens LoRaWan	31
Figura 6 – Processo de encriptação do Ascon-AEAD128	36
Figura 7 – Processo de dencriptação do Ascon-AEAD128	36
Figura 8 – Diagrama de sequência UML do fluxo principal	45
Figura 9 – Configuração de medição de corrente via resistor <i>shunt</i> e osciloscópio .	58
Figura 10 – Arquitetura especificada do sistema proposto.	60

Lista de tabelas

Tabela 1 – Cobertura dos campos relevantes no dataset Tour Perret	47
Tabela 2 – Variabilidade de RSSI por dispositivo no dataset Tour Perret	47

Lista de abreviaturas e siglas

ABNT	Associação Brasileira de Normas Técnicas
ABP	Activation by Personalization
ADR	Adaptive Data Rate
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
API	Application Programming Interface
ARIMA	AutoRegressive Integrated Moving Average
DDoS	Distributed Denial of Service
DES	Data Encryption Standard
DoS	Denial of Service
DRBG	Deterministic Random Bit Generator
ESP32	Espressif 32
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
KAT	Known Answer Tests
LoRa	Long Range
LoRaWAN	Long Range Wide Area Network
LPWAN	Low-Power Wide-Area Network
LSTM	Long Short-Term Memory
MAC	Medium Access Control
MAE	Mean Absolute Error
ML	Machine Learning
NIST	National Institute of Standards and Technology

OTAA	Over-the-Air Activation
RAM	Random Access Memory
RBG	Random Bit Generator
RF	Rádio Frequência
RMSE	Root Mean Squared Error
RNN	Recurrent Neural Network
RSSI	Received Signal Strength Indicator
SNR	Signal-to-Noise Ratio
TinyML	Tiny Machine Learning
TRNG	True Random Number Generator

Lista de símbolos

K	Chave criptográfica derivada
U	Sequência de bits do gerador aleatório aprovado
V	Sequência de bits independente usada na derivação
$\oplus$	Operação XOR (ou-exclusivo)
λ	Tamanho da <i>tag</i> de autenticação (em bits)
$\leq$	Relação menor ou igual

Sumário

1	Introdução	21
1.1	Motivação	21
1.2	Objetivos	23
1.3	Justificativa	23
1.4	Organização do Trabalho	23
2	Aspectos Conceituais	25
2.1	Redes IoT e o Protocolo LoRaWAN	25
2.1.1	LoRa e LoRaWAN: fundamentos e evolução do protocolo	25
2.1.2	Arquitetura da Rede	26
2.1.3	Ativação do Dispositivo	28
2.1.4	Transferência de Dados	29
2.1.5	Gerenciamento da Rede	31
2.1.6	LoRaWAN versus Wi-Fi no contexto do projeto	32
2.1.7	Implicações para a arquitetura proposta	33
2.2	Criptografia Leve (Lightweight Cryptography) e o Padrão Ascon	34
2.2.1	O Padrão Ascon-AEAD128	35
2.2.2	Comparativo de Algoritmos Criptográficos para IoT	36
2.3	Machine Learning	37
2.3.1	Detecção de Anomalias como Problema de Regressão Temporal	37
2.3.1.1	Contexto Prático dos Datasets Disponíveis	38
2.3.2	Modelos de Regressão para Séries Temporais	39
2.3.3	Considerações sobre Inferência em Dispositivos de Borda	41
3	Método do trabalho	43
3.1	Procedimento de definição da arquitetura do sistema	43
3.1.1	Critérios adotados na decomposição arquitetural	44
3.1.2	Modelagem das interações e do fluxo de dados	44
3.2	Especificação de Requisitos e Revisão Bibliográfica	45
3.2.1	Materiais e Bases de Dados	45
3.2.1.1	Dataset Tour Perret — base principal de modelagem	46
3.2.1.2	Análise Exploratória do Dataset Tour Perret	46
3.2.1.3	Dataset GUARD — papel complementar e de referência	48
3.2.1.3.1	Caracterização dos Ataques em GUARD	48
3.2.1.3.2	Estratégias de Validação com GUARD	48
3.2.2	Seleção de Variáveis	50

3.2.3	Procedimentos de Treinamento e Avaliação	51
3.2.3.1	Preparação e Divisão dos Dados	51
3.2.3.2	Treinamento dos Modelos	51
3.2.3.3	Avaliação da Qualidade de Regressão	52
3.2.3.4	Avaliação da Capacidade de Detecção de Anomalias	52
3.2.4	Procedimentos de Treinamento e Avaliação	53
3.2.5	Biblioteca Ascon	53
3.3	Atividades propostas para desenvolvimento e validação	54
3.3.1	Integração progressiva da infraestrutura	54
3.3.2	Atividades propostas de validação	55
3.3.3	Relação com o cronograma do projeto	55
3.4	Testes e Avaliação de Desempenho (<i>Benchmarking</i>)	55
3.4.1	<i>Known Answer Tests</i> (KAT)	55
3.4.2	Validação dos Dados Associados (AEAD)	56
3.4.3	Vazamento de Memória e Ataques de Canal Colateral	56
3.4.4	Avaliação de Consumo Energético	57
4	Especificação de Requisitos	59
4.1	Especificação da arquitetura do sistema	59
4.1.1	Requisitos funcionais da infraestrutura proposta	60
4.1.2	Requisitos não funcionais da infraestrutura proposta	61
4.2	Requisitos do Sistema de Detecção de Anomalias	61
4.2.1	Requisitos Funcionais	62
4.2.2	Requisitos Não Funcionais	62
4.3	Requisitos de Segurança do Sistema criptográfico	62
4.3.1	Segurança do Ascon-AEAD128	62
4.3.1.1	Requisitos funcionais	63
4.3.1.2	Requisitos não funcionais	64
5	Desenvolvimento do Trabalho	65
5.1	Atividades Previstas	65
5.1.1	Implementação da infraestrutura	65
5.1.2	Segurança com Ascon	65
5.1.3	Modelo de Machine Learning	66
5.1.4	Integração progressiva da solução	68
6	Considerações Preliminares	69
6.1	Perspectivas de Continuidade	69
6.1.1	Segurança Física do ESP32	70

REFERÊNCIAS 71

1 Introdução

A Internet das Coisas (IoT) é composta por dispositivos embarcados que desempenham funções específicas, atuando como sensores e controladores para uma variedade de aplicações tecnológicas produzidas hoje em dia, desde os aparelhos mais simples do cotidiano até processos industriais complexos. Com a evolução do hardware, os dispositivos IoT tornaram-se menores, mais rápidos, mais eficientes e com menor custo, o que levou a uma explosão do seu uso ([AL-FUQAHA et al., 2015](#)).

Contudo, um dos principais entraves para essa tecnologia decorre das limitações energéticas e computacionais desses dispositivos ([MCKAY et al., 2017](#)). Apesar da evolução vista nas últimas décadas, a IoT ainda enfrenta gargalos em processos que demandam alto poder computacional, devido à grande carga de processamento e ao elevado custo energético.

Diante desse cenário, surgem questões de segurança que podem ser exploradas para invadir e comprometer o funcionamento da rede. Para resolver a questão da transmissão segura e eficiente, padrões como o ASCON, pela necessidade de se padronizar e produzir um conjunto de métodos criptográficos que sejam leves para uso em dispositivos com baixo consumo energético ([TURAN et al., 2025](#)). Em paralelo, para viabilizar a comunicação de longa distância com baixa energia, utiliza-se amplamente o protocolo LoRaWAN, que possui aplicações fundamentais em diversas áreas, desde a infraestrutura de cidades inteligentes e controle de tráfego até o monitoramento de desastres, agricultura de precisão e automação industrial com foco em manutenção preditiva. O grande problema enfrentado por esse protocolo, no entanto, é a segurança da informação frente a ataques como jamming, replay attacks, spoofing e denial of service ([HESSEL; ALMON; HOLLICK, 2023](#)). A vulnerabilidade a ataques podem causar interrupções críticas e prejuízos severos, tornando indispensável a aplicação de métodos robustos de prevenção.

1.1 Motivação

A necessidade de operar com baixo consumo energético eleva bastante a complexidade do uso de dispositivos IoT, uma vez que tentativas de mitigar esse problema podem resultar em simplificações que geram problemas de segurança ([MCKAY et al., 2017](#)) ([HESSEL; ALMON; HOLLICK, 2023](#)). Diante disso, torna-se pertinente compreender a natureza desses ataques e suas formas de mitigação, além de estudar a eficiência e a eficácia de modelos de Machine Learning na detecção de intrusões e anomalias.

Um desses problemas é a vulnerabilidade a ataques diretos, como ataques de negação

de serviço (DDoS) ou Jamming, que podem esgotar a bateria do dispositivo ou impedir que a coleta de dados seja feita de maneira adequada (HESSEL; ALMON; HOLLICK, 2023). Outro grande desafio está na comunicação: é preciso garantir que a informação transmitida entre os dispositivos não possa ser interceptada nem modificada, assegurando a integridade, confidencialidade e autenticidade dos dados. Como os algoritmos de segurança comumente utilizados exigem um grande poder computacional, torna-se necessário buscar maneiras mais eficientes de proteger os dados durante a transmissão.

O uso de *Machine Learning* neste trabalho decorre da necessidade de detectar anomalias em redes LoRaWAN a partir de padrões temporais e comportamentais dos dados. Em contextos reais de IoT, abordagens baseadas apenas em regras fixas ou limiares estáticos tendem a ser limitadas. O comportamento normal pode variar conforme o contexto operacional (dispositivo, gateway, ambiente) e evoluir ao longo do tempo, tornando limiares fixos (ex: “se RSSI < -120 dBm, alertar”) inadequados. Além disso, métodos tradicionais apresentam dificuldade em capturar estruturas complexas em dados sequenciais, tais como degradação gradual de equipamento ou padrões não lineares de transmissão. Por outro lado, abordagens baseadas em aprendizado conseguem modelar relações temporais, identificar desvios sutis e adaptar-se dinamicamente à evolução do comportamento normal, mostrando-se mais adequadas para este contexto (CHALAPATHY; CHAWLA, 2019).

Por isso, a modelagem é realizada com base no comportamento esperado do dispositivo em suas circunstâncias normais de operação, buscando aprender padrões legítimos de funcionamento a partir dos dados observados. Nessa abordagem, desvios suficientemente relevantes em relação ao comportamento previamente aprendido são tratados como indícios de anomalia. Trata-se, portanto, de uma abordagem adaptativa, pois o modelo se ajusta ao perfil comportamental de cada nó monitorado. (HAYASHI; RUGGIERO, 2022).

Assim, neste trabalho, *Machine Learning* é tratado como um dos núcleos centrais da investigação. Seu papel não é apenas classificar dados, mas aprender perfis de operação, comparar abordagens de detecção e oferecer uma base experimental para uma etapa posterior de avaliação em borda. Essa decisão amplia o escopo da análise, permitindo estudar simultaneamente eficácia de detecção, robustez metodológica e viabilidade de implantação em borda.

O padrão Ascon, por sua vez, foi selecionado como a primeira escolha como novo padrão da competição de criptografia leve do NIST (NIST (2021)) assim como a primeira escolha de criptografia leve da competição CAESAR (CAESAR Committee, 2014). A relevância desse padrão se torna clara quando analisado seu baixo custo energético, rapidez, e confiabilidade, garantindo segurança para dados AEAD e ataques de canal colateral (DOBRAUNIG et al., 2023a).

1.2 Objetivos

O objetivo geral deste trabalho é implementar e avaliar um sistema de comunicação baseado no protocolo LoRaWAN, integrando a aplicação de modelos de *Machine Learning* e o padrão de criptografia leve Ascon. A finalidade central é analisar o nível de segurança e a resiliência que a combinação dessas duas tecnologias pode oferecer ao ecossistema de Internet das Coisas (IoT).

Para o alcance do objetivo geral, foram definidos os seguintes objetivos específicos:

- Treinar e testar modelos de *Machine Learning*, utilizando os *datasets* selecionados, para atuar na identificação e mitigação de ameaças ativas, tais como *jamming*, *spoofing* e ataques de Negação de Serviço (DoS).
- Implementar os algoritmos da família Ascon no nó sensor para estabelecer uma infraestrutura que garanta a privacidade, a autenticidade e a integridade dos dados transmitidos, com baixo custo computacional.
- Desenvolver o protótipo do sistema integrando as camadas de comunicação (LoRaWAN), de detecção e criptografia.
- Analisar o nível de segurança global oferecido pelo conjunto, validando a eficácia da solução proposta frente às restrições do hardware - em termos de consumo energético.

1.3 Justificativa

A relevância deste trabalho evidencia-se pelo papel central e pela rápida expansão dos dispositivos de Internet das Coisas (IoT) no cotidiano, com projeções indicando dezenas de bilhões de equipamentos conectados nos próximos anos (PANDEY; DIWAN; DHAR, 2025). Propor uma metodologia que integra tecnologias e conceitos críticos como o protocolo de comunicação LoRaWAN, modelos de *Machine Learning* e segurança criptográfica, e verificar a sua eficácia em cenários práticos, demonstra como a união dessas ferramentas pode alavancar a resiliência de sistemas reais, tornando-os cada vez mais robustos contra ameaças.

1.4 Organização do Trabalho

Para facilitar a leitura e a compreensão da pesquisa, este documento está estruturado da seguinte forma:

O Capítulo 2 apresenta os aspectos conceituais que fundamentam o projeto. Nele, são abordados o funcionamento do protocolo LoRaWAN e o paradigma da criptografia

leve (*Lightweight Cryptography*), detalhando o processo de padronização do NIST que elegeu a família Ascon, além de uma comparação com outras famílias de algoritmos. O capítulo também explora a base teórica de *Machine Learning* voltada para a detecção de anomalias.

O Capítulo 3 descreve o método do trabalho. Primeiramente, apresenta-se a abordagem de elicitación de requisitos adotada, incluindo análise de literatura, identificação de restrições de hardware e validação de viabilidade técnica. Em seguida, são detalhados os datasets analisados e selecionados para o modelo de Machine Learning, os procedimentos de treinamento e avaliação, bem como a justificativa para a adoção da biblioteca ascon-suite na implementação do padrão Ascon. O capítulo abrange ainda as etapas de configuração do ambiente, implementação e métodos de teste.

O Capítulo 4 trata da especificação de requisitos do sistema, definindo as restrições computacionais, de memória e de consumo energético que o protótipo deve respeitar para operar na borda da rede.

O Capítulo 5 é dedicado ao projeto e desenvolvimento. Nesta etapa, são descritas as tecnologias e ferramentas práticas utilizadas, a arquitetura montada, os testes realizados e a discussão dos resultados obtidos.

Por fim, o Capítulo 6 apresenta as considerações finais, resumizando as conclusões extraídas da pesquisa e apontando direcionamentos para futuras implementações e melhorias que podem surgir a partir deste trabalho.

2 Aspectos Conceituais

A proteção efetiva de um sistema não é alcançada por uma única técnica, mas pela combinação de estratégias em múltiplas camadas. Conforme o *NIST Cybersecurity Framework* (NIST, 2024), a segurança robusta exige tanto **prevenção** (impedindo que ataques ocorram) quanto **detecção** (identificando atividades suspeitas). Este capítulo apresenta os conceitos que fundamentam este trabalho dentro desse paradigma: protocolos de comunicação para IoT, criptografia leve como mecanismo preventivo, e Machine Learning como mecanismo de detecção comportamental.

2.1 Redes IoT e o Protocolo LoRaWAN

As redes de Internet das Coisas (IoT) utilizam amplamente o protocolo LoRaWAN em cenários que exigem baixo consumo de energia e ampla cobertura. Embora o LoRaWAN apresente certa maturidade e conte com mecanismos de proteção continuamente aprimorados, ataques e problemas de segurança ainda afetam sua resiliência. Consequentemente, permanece necessária a evolução contínua de técnicas de prevenção e mitigação dessas ameaças (HESSEL; ALMON; HOLLICK, 2023).

2.1.1 LoRa e LoRaWAN: fundamentos e evolução do protocolo

O LoRaWAN se destaca como uma tecnologia voltada a redes LPWAN (*Low-Power Wide-Area Network*), sendo utilizado em aplicações como monitoramento remoto, agricultura inteligente e sistemas distribuídos com dispositivos embarcados. Sua proposta é permitir a comunicação entre dispositivos com baixo consumo energético, ampla cobertura e custo operacional reduzido.

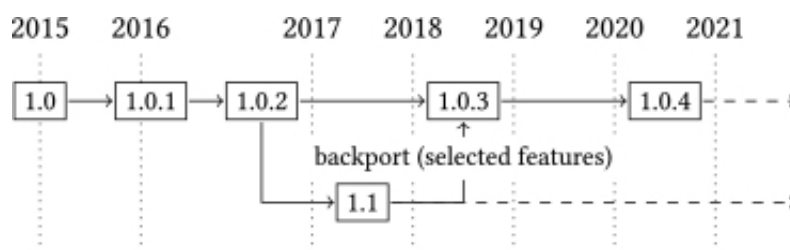
É importante distinguir os termos *LoRa* e *LoRaWAN*, que frequentemente são tratados como sinônimos, embora representem camadas diferentes da tecnologia. O LoRa corresponde à camada física da comunicação, isto é, à técnica de modulação empregada no enlace sem fio. Já o LoRaWAN define o protocolo de comunicação acima dessa camada, estabelecendo regras para organização da rede, troca de mensagens e controle da comunicação. Em outras palavras, o LoRa fornece o meio de transmissão por rádio, enquanto o LoRaWAN define o funcionamento da rede sobre esse meio.

De forma geral, o LoRaWAN foi projetado para redes compostas por dispositivos com recursos limitados, muitas vezes alimentados por bateria e instalados em locais de difícil manutenção. Por isso, o protocolo prioriza simplicidade operacional, baixo consumo e comunicação eficiente, ainda que com taxas de transmissão menores do que tecnologias

voltadas a maior vazão de dados. Essa característica o torna adequado para cenários em que pequenas quantidades de informação precisam ser transmitidas periodicamente, sem necessidade de comunicação contínua.

A evolução do LoRaWAN também é um aspecto importante. Desde sua primeira publicação, em 2015, o protocolo passou por revisões que resultaram em dois ramos principais: a família 1.0.x e a versão 1.1. As versões da família 1.0.x mantiveram interoperabilidade entre si e trouxeram principalmente correções, esclarecimentos e ajustes incrementais. Já a versão 1.1 introduziu mudanças mais relevantes, especialmente na organização de funções da rede e nos mecanismos de segurança, tornando mais clara a separação entre responsabilidades de rede e de aplicação.

Figura 1 – Histórico de versionamento da especificação da LoRaWAN



Fonte: (HESSEL; ALMON; HOLLICK, 2023)

Mesmo com a introdução de melhorias nas versões mais recentes, a adoção de novas revisões do protocolo não ocorre de forma imediata em sistemas reais. Em aplicações IoT, sensores podem permanecer em operação por muitos anos, frequentemente sem atualização de firmware. Como consequência, versões anteriores do LoRaWAN continuam relevantes na prática, e essa coexistência entre gerações do protocolo precisa ser considerada em qualquer análise técnica ou experimental.

Assim, compreender a distinção entre LoRa e LoRaWAN, bem como a evolução do protocolo ao longo de suas versões, é essencial para analisar corretamente seu funcionamento em sistemas embarcados conectados. A partir dessa base, as próximas subseções detalham a arquitetura da rede, o processo de ativação dos dispositivos, a transferência de dados, os mecanismos de gerenciamento da comunicação e a relação dessas tecnologias com a arquitetura proposta neste trabalho.

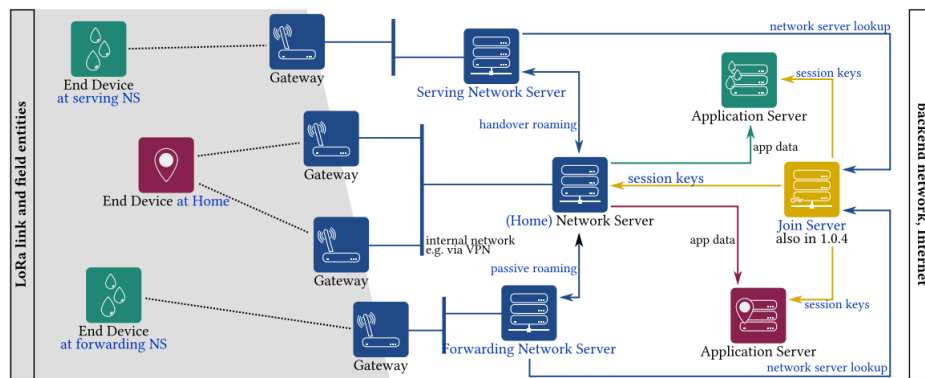
2.1.2 Arquitetura da Rede

A arquitetura do LoRaWAN é composta por diferentes entidades, cada uma com uma função específica no funcionamento da rede. De forma geral, os dispositivos finais, chamados de *End Devices*, são responsáveis por coletar e transmitir dados. Esses dispositivos se comunicam por rádio com um ou mais *Gateways*, que atuam como pontes entre a rede

LoRa e a infraestrutura conectada por IP. Os gateways, por sua vez, apenas recebem os quadros transmitidos pelos dispositivos e os encaminham para os servidores da rede, sem realizar processamento completo do protocolo. (HESSEL; ALMON; HOLLIICK, 2023)

Essa organização forma uma topologia conhecida como *star-of-stars*. Nela, os dispositivos finais não se comunicam diretamente entre si, e também não estabelecem vínculo fixo com um único gateway. Em vez disso, uma mesma transmissão pode ser recebida por vários gateways que estejam dentro de alcance. Essa característica aumenta a cobertura da rede e oferece maior robustez na recepção das mensagens, especialmente em cenários distribuídos e com mobilidade.

Figura 2 – Arquitetura de rede LoRaWAN



Fonte: (HESSEL; ALMON; HOLLIICK, 2023)

Após o encaminhamento feito pelos gateways, as mensagens chegam ao *Network Server*. Esse servidor exerce papel central na rede, pois é responsável por processar as mensagens recebidas, eliminar duplicidades causadas pela recepção do mesmo quadro por múltiplos gateways, validar a integridade das mensagens e executar as funções de gerenciamento da camada MAC. Em outras palavras, é o *Network Server* que coordena o funcionamento lógico da rede LoRaWAN.

Os dados de aplicação seguem então para o *Application Server*, que é o componente responsável por tratar a informação útil transportada pelo dispositivo final. É nesse servidor que os dados recebidos podem ser decifrados, interpretados e encaminhados para a aplicação de interesse, como um sistema de monitoramento, um banco de dados ou uma plataforma de análise. Essa separação entre funções de rede e funções de aplicação é importante porque organiza melhor as responsabilidades do sistema e reduz o acoplamento entre comunicação e processamento da informação.

Na evolução do protocolo, especialmente a partir do LoRaWAN 1.1, o *Join Server* passa a ter um papel mais explícito na arquitetura. Esse componente participa do processo de ativação dos dispositivos e da derivação de chaves de sessão, contribuindo para uma

separação mais clara entre gerenciamento de rede e segurança da aplicação. Com isso, a arquitetura do LoRaWAN se torna mais organizada do ponto de vista funcional e mais adequada a cenários em que diferentes operadores ou domínios de serviço precisam cooperar.

Assim, a arquitetura do LoRaWAN pode ser entendida como uma organização em camadas funcionais: o dispositivo final coleta e transmite os dados, o gateway encaminha os quadros recebidos, o *Network Server* gerencia a comunicação da rede, e o *Application Server* entrega os dados à aplicação. Nas versões mais recentes, o *Join Server* complementa essa estrutura ao apoiar o processo de ativação e gerenciamento de chaves. Essa divisão de papéis é fundamental para compreender como o protocolo opera e como seus componentes podem ser relacionados à arquitetura proposta neste trabalho.

2.1.3 Ativação do Dispositivo

Antes de transmitir dados de aplicação, um dispositivo LoRaWAN precisa ser ativado na rede. Esse processo define como o dispositivo passa a ser reconhecido pelo sistema e como obtém o contexto necessário para a comunicação, incluindo identificadores, parâmetros de sessão e chaves utilizadas ao longo da operação. De forma geral, o LoRaWAN admite dois modos principais de ativação: *Activation By Personalization* (ABP) e *Over-The-Air Activation* (OTAA). (HESSEL; ALMON; HOLLICK, 2023)

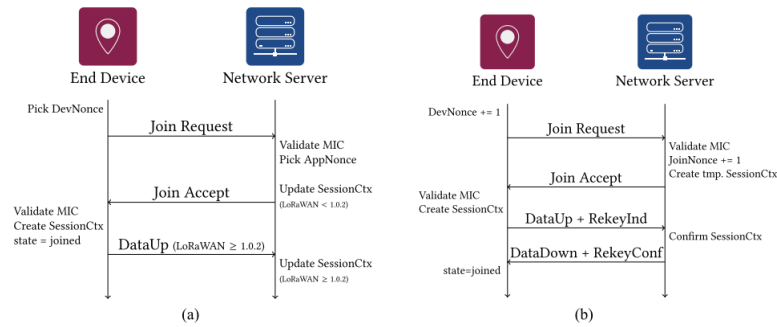
No modo ABP, o dispositivo já é configurado previamente com os parâmetros de sessão necessários para operar. Assim, ele pode iniciar a transmissão diretamente, sem realizar um procedimento de ingresso na rede a cada inicialização. Essa abordagem é mais simples do ponto de vista de implementação, mas oferece menor flexibilidade operacional, já que os parâmetros de comunicação ficam previamente fixados no dispositivo.

No modo OTAA, por outro lado, o dispositivo realiza um procedimento de ingresso na rede antes de iniciar sua operação normal. Nesse processo, o nó envia uma solicitação de ativação e, a partir da resposta da infraestrutura da rede, passa a operar com parâmetros e chaves de sessão derivados especificamente para aquela conexão. Em comparação com o ABP, essa abordagem é mais flexível e mais adequada a cenários reais, pois facilita renovação de chaves, adaptação do contexto de comunicação e suporte a funcionalidades mais avançadas do protocolo.

De forma simplificada, a ativação por OTAA envolve a troca de mensagens de ingresso e a geração de chaves de sessão a partir de credenciais previamente associadas ao dispositivo. Esse mecanismo permite que a rede estabeleça uma nova sessão de comunicação de forma controlada, em vez de depender sempre de parâmetros estáticos. Por esse motivo, o OTAA tende a ser considerado a alternativa mais robusta para implantações práticas, especialmente quando se deseja maior controle do ciclo de vida dos dispositivos e melhor

suporte a segurança.

Figura 3 – (a) OTAA processo de entrada LoRaWAN 1.0.x. (b) OTAA processo de entrada LoRaWAN 1.1



Fonte: (HESSEL; ALMON; HOLLICK, 2023)

A evolução do protocolo também impacta esse processo. Nas versões da família 1.0.x, a ativação já podia ocorrer por ABP ou OTAA, mas a versão 1.1 introduziu uma separação mais clara entre funções de rede e aplicação, além de mudanças no tratamento das chaves e no papel do *Join Server*. Com isso, o processo de ativação passou a ser mais estruturado e alinhado com uma arquitetura de segurança mais organizada.

Assim, a ativação do dispositivo representa uma etapa fundamental no funcionamento do LoRaWAN, pois estabelece as condições iniciais para que a comunicação ocorra de forma válida dentro da rede. Enquanto o ABP oferece uma alternativa mais simples, o OTAA apresenta maior flexibilidade e melhor aderência a cenários práticos e escaláveis. A partir dessa etapa, o dispositivo pode iniciar a troca de dados com a infraestrutura da rede, conforme discutido na próxima subseção.

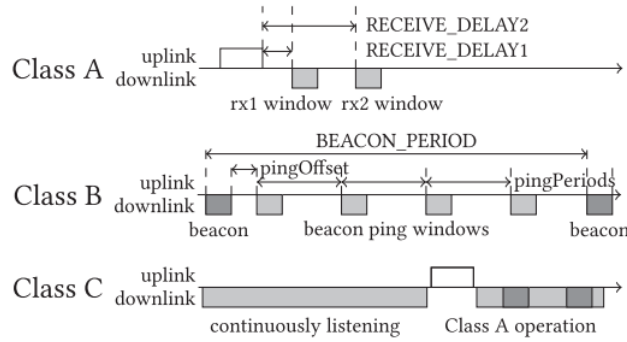
2.1.4 Transferência de Dados

Após a ativação do dispositivo, a comunicação de dados pode ocorrer entre o dispositivo final e a infraestrutura da rede. No LoRaWAN, as mensagens enviadas pelo dispositivo em direção à rede são chamadas de *DataUp*, enquanto as mensagens enviadas pela rede ao dispositivo são chamadas de *DataDown*. Essa troca de dados pode ocorrer de forma simples, apenas com envio da informação, ou com solicitação de confirmação de recebimento, dependendo da necessidade da aplicação. (HESSEL; ALMON; HOLLICK, 2023)

Quando não há necessidade de confirmação, utilizam-se mensagens não confirmadas, que reduzem o overhead da comunicação e tendem a ser mais adequadas a aplicações periódicas de monitoramento. Já nas mensagens confirmadas, a camada MAC pode exigir uma resposta da rede, permitindo ao dispositivo verificar se a transmissão foi reconhecida.

Essa distinção é importante porque envolve um compromisso entre confiabilidade e consumo de recursos, já que confirmações adicionais aumentam o uso do canal e o gasto energético.

Figura 4 – Classe para comunicacoes ED



Fonte: (HESSEL; ALMON; HOLLICK, 2023)

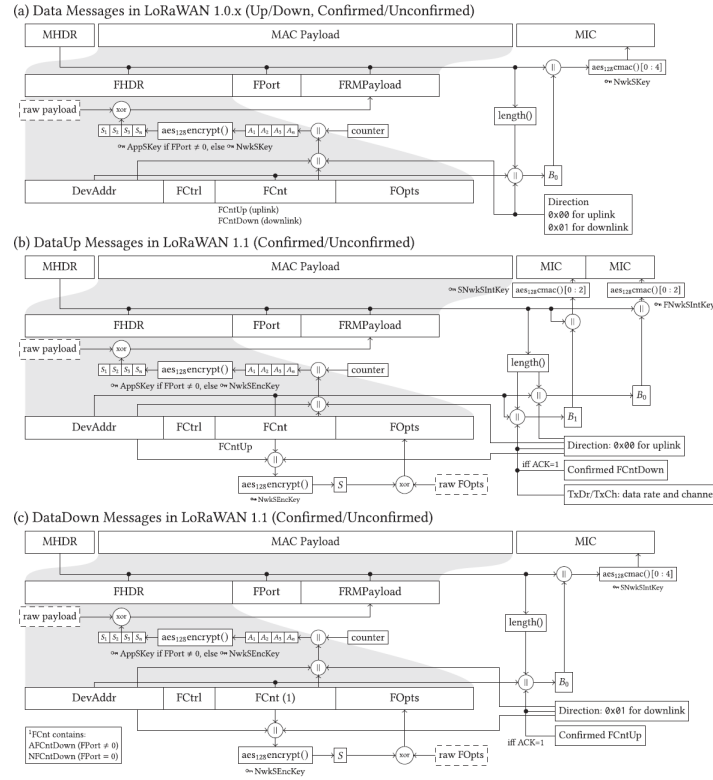
O LoRaWAN define três modos principais de operação para os dispositivos finais, chamados de classes A, B e C. A Classe A é a padrão e está presente em todos os dispositivos LoRaWAN. Nela, o dispositivo inicia a comunicação por meio de um *uplink* e, após essa transmissão, abre duas pequenas janelas de recepção para possíveis mensagens de *downlink*. Essas janelas são conhecidas como RX1 e RX2. Esse modelo é o mais eficiente em termos energéticos, pois o dispositivo permanece a maior parte do tempo fora de escuta.

Na Classe B, além do comportamento básico da Classe A, o dispositivo sincroniza sua operação com sinais periódicos da rede e passa a abrir janelas adicionais de recepção em instantes programados. Isso permite maior previsibilidade para o envio de mensagens em *downlink*, embora aumente a complexidade da operação. Já na Classe C, o dispositivo permanece praticamente em escuta contínua, fechando a recepção apenas quando transmite. Essa abordagem reduz a latência de *downlink*, mas aumenta significativamente o consumo de energia, sendo mais adequada a dispositivos com menor restrição energética.

As mensagens de dados do LoRaWAN possuem uma estrutura padronizada. De forma geral, elas incluem um cabeçalho com a identificação do tipo de mensagem, um cabeçalho de quadro com informações de controle e endereçamento, um contador de quadros e o campo destinado à carga útil da aplicação. Entre esses elementos, o contador de quadros, representado por *FCnt*, é especialmente importante para o acompanhamento da sequência das mensagens. Outros campos, como *FPort* e *FOpts*, ajudam a distinguir dados de aplicação e informações de controle da rede. Além disso, as mensagens incluem um código de integridade, chamado *MIC*, utilizado para verificação da validade da comunicação.

De modo geral, a transferência de dados no LoRaWAN foi projetada para equilibrar simplicidade, baixo consumo e capacidade de operação em redes distribuídas de longo

Figura 5 – Estrutura e Criptografia das mensagens LoRaWAN



Fonte: (HESSEL; ALMON; HOLLICK, 2023)

alcance. A existência de diferentes classes de operação e de mecanismos para mensagens confirmadas ou não confirmadas permite adaptar o comportamento da comunicação às necessidades da aplicação. A partir dessa base, torna-se possível compreender melhor como a rede ajusta e gerencia sua operação, tema discutido na próxima subseção.

2.1.5 Gerenciamento da Rede

Como o LoRaWAN é um protocolo de camada MAC voltado a redes LPWAN, ele precisa de mecanismos para configurar dispositivos, ajustar parâmetros de comunicação e acompanhar o estado da rede. Parte dessas funções já aparece diretamente nos próprios campos das mensagens, como endereços, contadores de quadros e confirmações. No entanto, o protocolo também define comandos específicos para gerenciamento, conhecidos como *MAC commands*, que permitem trocar informações de controle entre o dispositivo final e a infraestrutura da rede. (HESSEL; ALMON; HOLLICK, 2023)

Esses comandos são usados para diferentes finalidades de operação. Entre elas, estão a obtenção de informações sobre a qualidade da conexão, o ajuste de parâmetros de transmissão e a sincronização de tempo. Assim, o gerenciamento da rede não ocorre como uma função separada do restante do protocolo, mas sim como parte do próprio fluxo de

comunicação entre os dispositivos e os servidores da rede.

Entre os mecanismos de gerenciamento, um dos mais importantes é o *Adaptive Data Rate* (ADR). O ADR tem como objetivo ajustar a taxa de transmissão de dados e outros parâmetros de rádio de acordo com as condições de comunicação observadas na rede. Na prática, isso permite definir combinações mais adequadas de *Spreading Factor*, largura de banda e potência de transmissão, buscando equilibrar confiabilidade, consumo de energia e uso eficiente do meio de comunicação.

De forma geral, o gerenciamento da rede no LoRaWAN existe para manter a comunicação funcional e eficiente ao longo do tempo. Por meio de comandos de controle e mecanismos como o ADR, o protocolo consegue ajustar seu comportamento às condições reais de operação, o que é especialmente importante em ambientes distribuídos, sujeitos a variações de alcance, interferência e disponibilidade energética. Com essa base conceitual, torna-se possível avançar para uma análise mais voltada à forma como essas tecnologias serão utilizadas na arquitetura proposta neste trabalho.

2.1.6 LoRaWAN versus Wi-Fi no contexto do projeto

Embora LoRaWAN e Wi-Fi sejam tecnologias de comunicação sem fio, elas atendem a necessidades diferentes e, neste trabalho, não são tratadas como soluções concorrentes. Cada uma delas ocupa um papel específico na arquitetura proposta, de acordo com suas características de alcance, consumo energético, taxa de transmissão e facilidade de integração com os demais componentes do sistema.

O LoRaWAN foi pensado para aplicações de longo alcance e baixo consumo, sendo mais adequado a dispositivos embarcados que operam com recursos limitados e transmitem pequenas quantidades de dados de forma periódica ou eventual. Essas características o tornam compatível com o cenário deste projeto, no qual os nós sensores e atuadores precisam operar de forma eficiente, com comunicação sem fio e possibilidade de instalação em locais mais afastados do ponto central da rede.

O Wi-Fi, por outro lado, oferece maior taxa de transmissão e integração mais direta com redes IP e serviços de aplicação, como APIs e servidores de armazenamento. Em contrapartida, apresenta maior consumo energético e menor adequação a nós distribuídos que precisem operar por longos períodos com restrição de energia. Por esse motivo, o Wi-Fi não foi adotado como tecnologia principal de comunicação entre os nós embarcados e a infraestrutura de campo.

No contexto deste trabalho, a solução adotada combina as duas tecnologias de forma complementar. O trecho entre os nós sensores/atuadores e o gateway utiliza LoRaWAN, preservando as vantagens de alcance e baixo consumo no nível embarcado. Já o trecho entre o gateway e o servidor utiliza Wi-Fi, aproveitando sua integração mais simples com

a infraestrutura de rede e com os serviços responsáveis por receber, armazenar e processar os dados coletados.

Essa divisão permite separar dois problemas distintos dentro da arquitetura. O primeiro é a comunicação entre dispositivos embarcados em campo, em que baixo consumo e cobertura são requisitos centrais. O segundo é a entrega dos dados para uma camada de processamento e persistência, em que a prioridade passa a ser conectividade com a rede local ou com a Internet, além da facilidade de integração com a aplicação. Assim, a escolha conjunta de LoRaWAN e Wi-Fi não representa uma sobreposição de tecnologias, mas sim uma composição coerente entre camadas com funções diferentes.

Dessa forma, o uso de LoRaWAN no trecho embarcado e de Wi-Fi no trecho de retaguarda estabelece uma arquitetura compatível com os objetivos do projeto. Ao mesmo tempo em que preserva as vantagens de uma rede LPWAN na comunicação entre os dispositivos de campo, essa abordagem também facilita a construção da interface com o servidor, a persistência dos dados e a futura integração com mecanismos de análise e segurança.

2.1.7 Implicações para a arquitetura proposta

Os conceitos apresentados nas subseções anteriores permitem situar a arquitetura adotada neste trabalho. No projeto proposto, os nós sensores e atuadores operam como dispositivos finais responsáveis pela coleta de dados do ambiente e pela transmissão dessas informações pela rede sem fio. Essa comunicação ocorre no trecho embarcado por meio do LoRaWAN, explorando suas características de longo alcance e baixo consumo energético.

O gateway exerce o papel de ponto de transição entre a rede de campo e a infraestrutura de retaguarda. Sua função principal é receber os quadros enviados pelos dispositivos finais e encaminhá-los para a camada de processamento do sistema. A partir desse ponto, os dados podem ser entregues a um servidor ou API responsável por armazenamento, organização e disponibilização das informações para etapas posteriores de análise.

Essa divisão é importante porque separa o problema da comunicação embarcada do problema de processamento dos dados. No nível dos nós em campo, o foco está em transmitir com eficiência, respeitando restrições de energia, alcance e capacidade computacional. Já no nível do gateway e do servidor, torna-se possível concentrar funções de persistência, observabilidade e integração com serviços de aplicação, sem impor esse custo diretamente aos dispositivos finais.

Entretanto, o uso do LoRaWAN por si só não elimina todos os desafios de segurança do sistema. Como discutido na literatura, a proteção efetiva da rede depende não apenas do protocolo, mas também da forma como seus mecanismos são implementados e integrados à aplicação. Em particular, a presença de dados trafegando entre dispositivos embarcados,

gateway e infraestrutura de backend torna necessário considerar mecanismos adicionais de proteção, especialmente quando o sistema opera em ambientes distribuídos e sujeitos a diferentes tipos de ameaça.

Além disso, a própria dinâmica operacional da rede produz informações que podem ser úteis para análise de comportamento, identificação de falhas e detecção de atividades anômalas. Parâmetros de transmissão, padrões temporais de envio e eventos observados na comunicação podem servir como base para uma camada adicional de monitoramento do sistema, complementando a proteção oferecida pelos mecanismos tradicionais do protocolo.

Dessa forma, a arquitetura proposta neste trabalho não se limita ao transporte de dados entre dispositivos e servidor. Ela também cria as condições para incorporar camadas complementares de segurança e análise. Nesse contexto, a próxima seção discute o uso de criptografia leve como mecanismo adequado à proteção de dados em dispositivos com recursos restritos, enquanto a seção seguinte apresenta o papel do *Machine Learning* como apoio à análise do comportamento da rede e à detecção de anomalias.

2.2 Criptografia Leve (Lightweight Cryptography) e o Padrão Ascon

Em dispositivos IoT, onde o baixo consumo de energia é um requisito essencial para viabilizar operações em diversas áreas, as aplicações que lidam com dados sensíveis e exigem sigilo demandam a adoção de métodos criptográficos que sejam, simultaneamente, leves e seguros.

O Instituto Nacional de Padrões e Tecnologia dos Estados Unidos (NIST) atua, desde 1973, no desenvolvimento de normas para proteger dados sensíveis transmitidos entre dispositivos. Seu histórico abrange desde a criação do *Data Encryption Standard* (DES) e a consolidação do atual padrão de criptografia simétrica, o *Advanced Encryption Standard* (AES), até as recentes padronizações de criptografia pós-quântica (*Post-Quantum Cryptography*) e de criptografia leve (*Lightweight Cryptography*) (CHEN; SCHOLL, 2022).

Com o crescimento expressivo do uso de dispositivos IoT, a necessidade de estabelecer um padrão robusto de criptografia leve tornou-se urgente. Isso impulsionou o desenvolvimento de uma nova padronização internacional com o objetivo de contemplar as primitivas criptográficas e os modos de operação adequados para ambientes restritos. Os critérios exigidos para esse novo padrão incluíam resiliência de segurança, flexibilidade de implementação, baixa sobrecarga ao integrar múltiplas funções e proteção intrínseca contra ataques de canal lateral (*Side-Channel Attacks*), entre outros requisitos técnicos (MCKAY et al., 2017).

O processo de padronização promovido pelo NIST ocorreu ao longo de múltiplas fases de testes e escrutínio público. Inicialmente, 57 algoritmos foram submetidos para

consideração. Desses, 32 avançaram para a segunda rodada e, após rigorosas avaliações de desempenho e segurança, apenas 10 chegaram à fase final. A culminação desse extenso projeto ocorreu com o anúncio da família Ascon como a grande vencedora (NIST, 2017).

O desenvolvimento do padrão Ascon começou em 2014 e ele será o foco principal deste trabalho. Na prática, ele funciona como um conjunto de algoritmos construído sob medida para dispositivos com poucos recursos. A padronização oficial conta com quatro variações principais. A primeira delas é o Ascon-AEAD128, que criptografa os dados, verifica a autenticidade da mensagem e oferece uma boa proteção contra ataques de canal lateral. A segunda variação é o Ascon-Hash256, criado especificamente para garantir a integridade dos dados transmitidos pela rede. Por fim, existem o Ascon-XOF128 e o Ascon-CXOF128. Ambos funcionam como funções de saída extensível, uma característica que permite ajustar o tamanho do *hash* gerado e economizar recursos computacionais durante o processamento (NIST, 2025).

2.2.1 O Padrão Ascon-AEAD128

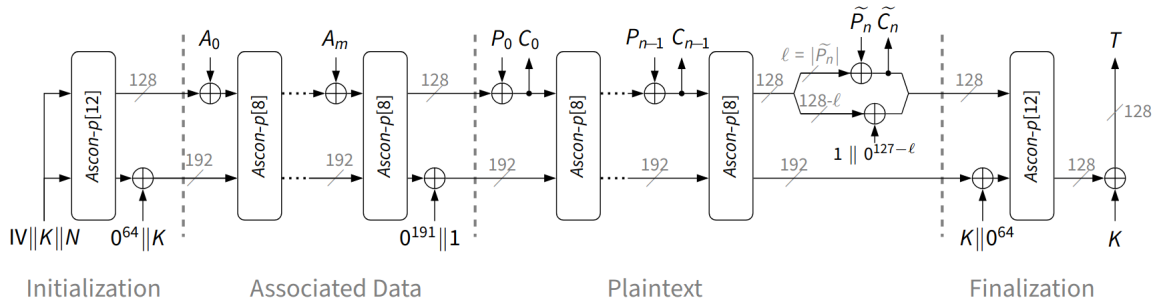
O algoritmo de encriptação escolhido para o projeto é o Ascon-AEAD128 NIST (2025). A utilização de funções de *hash* adicionais não será necessária, uma vez que esse algoritmo já garante confidencialidade, integridade e autenticidade em uma única operação de criptografia autenticada.

O processo de encriptação do Ascon-AEAD128 recebe como entrada uma chave secreta e um *nonce*, ambos de 128 bits, além dos dados associados (*Associated Data* - AD) e o texto em claro a ser cifrado (*plaintext*), que possuem tamanhos variáveis. A saída dessa função é o texto cifrado (*ciphertext*) acompanhado de uma *Tag* de Autenticação de 128 bits. Por sua vez, a função de decifração recebe a chave, o *nonce*, os dados associados, o texto cifrado e a *tag* de autenticação. O texto original em claro só é retornado se a *tag* for validada com sucesso. Caso contrário, a operação é abortada e retorna um erro (*fail*).

O estado interno inicial de ambos os processos possui 320 bits, sendo formado pela concatenação de 64 bits do Vetor de Inicialização (IV), 128 bits da chave secreta e 128 bits do *nonce*. Após a inicialização, o algoritmo segue em fases: primeiro ocorre o processamento dos dados associados (se presentes), seguido pela cifração do texto em blocos e, ao final de todo o processamento, a fase de finalização gera a *tag* de autenticação.

O processo de decifração possui uma estrutura muito semelhante à da encriptação. Ele passa pela mesma inicialização do estado de 320 bits e pelo processamento dos dados associados. A diferença principal ocorre nas etapas finais: o texto cifrado é processado bloco a bloco para recuperar o texto original e, na fase de finalização, a chave secreta é combinada com o estado interno para recalculer a *tag* de autenticação. Se a *tag* recalculada for idêntica à *tag* recebida, o texto decifrado é considerado autêntico e é retornado, caso

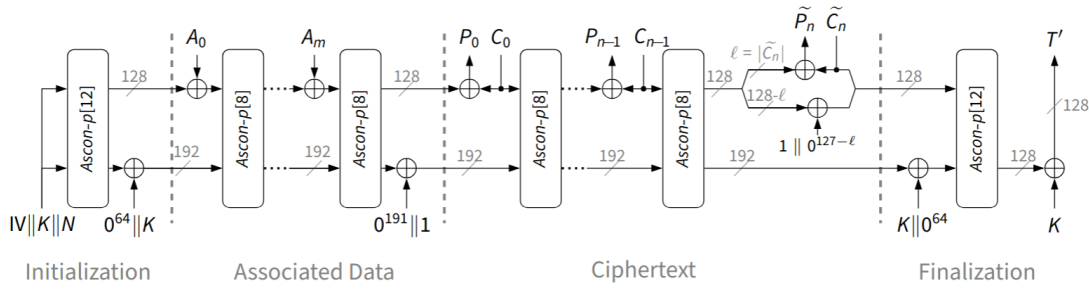
Figura 6 – Processo de encriptação do Ascon-AEAD128



Fonte: (NIST, 2025)

contrário, a mensagem é descartada.

Figura 7 – Processo de decryptação do Ascon-AEAD128



Fonte: (NIST, 2025)

2.2.2 Comparativo de Algoritmos Criptográficos para IoT

Durante a pesquisa inicial sobre criptografia leve, outros algoritmos foram avaliados, mas nenhum se mostrou tão adequado às necessidades do projeto quanto o padrão Ascon. Entre as principais alternativas analisadas, destacam-se o Speck [Baldi et al. \(2025\)](#) e o ChaCha20 [Bernstein \(2008\)](#). No entanto, a análise aprofundada revelou que nenhum destes se adequava às premissas metodológicas e práticas do projeto quanto o padrão Ascon e as soluções nativas em hardware.

O Speck é uma cifra de bloco pura e bastante leve. No entanto, ele possui uma limitação estrutural importante para redes modernas: não oferece Criptografia Autenticada com Dados Associados (AEAD) de forma nativa. Para garantir a integridade da mensagem, seria necessária a adição de um algoritmo de autenticação secundário e independente (como um código de autenticação de mensagem, MAC). O Speck teve sua padronização internacional ativamente rejeitada pela ISO (*International Organization for Standardization*) por falta de transparência em seu design ([MCCARTHY, 2018](#)).

O ChaCha20, por sua vez, é uma cifra de fluxo de altíssimo desempenho, amplamente adotada como padrão da indústria para cenários de software onde **não** há aceleração de hardware disponível. Contudo, assim como o Speck, ele carece de propriedades *AEAD* embutidas. A exigência extra de se acoplar o algoritmo Poly1305 consome memória e processamento. Como o microcontrolador utilizado neste trabalho (família ESP32) possui aceleração criptográfica nativa para AES, o cenário de uso principal do ChaCha20 perde o sentido prático.

Para a análise comparativa de viabilidade, este trabalho também optou por não focar nos demais algoritmos finalistas do processo de padronização do NIST (como TinyJAMBU ou Xoodoo) [NIST \(2021\)](#), uma vez que a superioridade do Ascon sobre seus pares já foi exaustivamente documentada e comprovada pelo próprio comitê no relatório oficial NIST IR 8454 ([TURAN et al., 2023](#)).

Dessa forma, a metodologia de comparação deste trabalho foi refinada para responder à questão prática sobre o embate entre o novo padrão de software e o padrão legado de hardware. Os testes focarão exclusivamente em comparar o Ascon (novo padrão *Lightweight Cryptography* do NIST) contra o AES com modo de operação autenticado (*AES-GCM* ou *AES-CCM*) rodando diretamente no coprocessador de hardware do microcontrolador. Essa abordagem permite avaliar se a migração para a criptografia leve em software traz vantagens reais de consumo, memória e latência frente ao uso do acelerador de silício já embutido nas placas comerciais utilizadas em redes *LoRaWAN*.

2.3 Machine Learning

O uso de *Machine Learning* é coerente com o estado da arte recente. Trabalhos como ([FARHAD et al., 2023](#)) demonstram que Machine Learning em LoRaWAN vem sendo explorado para otimizar parâmetros de rede (*spreading factor*, potência de transmissão) e classificar padrões de mobilidade.

Embora parte significativa da literatura esteja concentrada em desempenho e eficiência, esse mesmo conjunto de atributos também é extremamente relevante para tarefas de segurança e detecção de anomalias, já que comportamentos suspeitos frequentemente se refletem justamente em mudanças desses padrões operacionais ([FARHAD et al., 2023](#); [VANGELISTA et al., 2023](#)).

2.3.1 Detecção de Anomalias como Problema de Regressão Temporal

Em redes LoRaWAN operando em condições reais, os dispositivos finais transmitem mensagens de forma periódica ou orientada a evento, produzindo naturalmente sequências de observações ao longo do tempo. Grandezas como a intensidade do sinal recebido, a relação sinal-ruído e os contadores de quadros variam de forma contínua e apresentam

padrões de comportamento relativamente estáveis durante a operação normal da rede. Assim, desvios abruptos ou sistemáticos nesses padrões podem indicar a presença de falhas, interferências ou atividades maliciosas, tornando a análise temporal dessas variáveis uma abordagem promissora para a detecção de anomalias.

No entanto, a aplicação de técnicas supervisionadas de classificação enfrenta uma limitação estrutural importante nesse contexto. Em ambientes de operação real, dados rotulados que identifiquem explicitamente quadros legítimos e quadros associados a ataques são menos presentes.

O *dataset* Tour Perret, utilizado neste trabalho, é representativo da condição: trata-se de um conjunto de dados proveniente de sensores reais instalados na Tour Perret, em Grenoble, que registra o comportamento contínuo da rede sem a presença de rótulos de ataque. Essa característica inviabiliza a utilização direta de classificadores supervisionados e exige uma abordagem metodológica distinta - o aprendizado de padrões esperados. Mas devido a sua grande volumetria e qualidade geral de dados, esta fonte será utilizada como principal, para treinos não supervisionados, assim como descrito na seção 3.2.1.1.

A estratégia adotada neste trabalho baseia-se na modelagem do comportamento esperado das variáveis de rede ao longo do tempo, por meio de modelos de regressão para séries temporais. Nessa abordagem, o modelo é treinado exclusivamente sobre dados que representam a operação normal da rede, aprendendo os padrões temporais característicos de cada variável monitorada. Durante a fase de inferência, o modelo produz uma previsão do valor esperado para cada instante de tempo, e o resíduo entre o valor previsto e o valor observado é utilizado como indicador de anomalia. Observações cujo resíduo supere um limiar previamente definido (Como 3 desvios padrão da média) são sinalizadas como potencialmente anômalas. Esse princípio, conhecido na literatura como detecção de anomalias baseada em previsão, tem sido aplicado com sucesso em contextos de IoT e redes de sensores justamente por não depender da disponibilidade de dados rotulados de ataque (CHALAPATHY; CHAWLA, 2019).

2.3.1.1 Contexto Prático dos Datasets Disponíveis

A escolha da abordagem não supervisionada é também sustentada por limitações práticas do contexto. Para este trabalho, dois datasets públicos foram selecionados estrategicamente:

Tour Perret (Treinamento/Validação): Dados reais de operação normal, sem injeção ou simulação de ataques. Adequado para aprender padrões verdadeiros de comportamento esperado.

GUARD (Validação contra Ataques Conhecidos): Dados com ataques rotulados (replay, spoofing). Permite verificar se o modelo detecta desvios associados a ataques

já identificados.

Esta estratégia de dois datasets permite validação em duas fases: primeiro, verificar se o modelo aprendeu comportamento normal corretamente (Seção 3.2.3.3); segundo, verificar se detecta anomalias quando confrontado com ataques conhecidos (Seção 3.2.3.4). A descrição detalhada destes datasets é apresentada na Seção 3.2.1.

Essa escolha também apresenta uma vantagem interpretativa relevante para o contexto do trabalho. Ao modelar o comportamento normal da rede, torna-se possível não apenas detectar anomalias, mas também caracterizá-las em termos da magnitude e da persistência do desvio observado, oferecendo informações mais ricas do que uma simples classificação binária. Além disso, a abordagem por regressão é mais adaptável a variações graduais do comportamento da rede ao longo do tempo, como mudanças sazonais ou degradação natural do sinal, uma vez que o modelo pode ser periodicamente reatualizado com dados recentes de operação normal.

2.3.2 Modelos de Regressão para Séries Temporais

A escolha dos modelos de regressão utilizados neste trabalho foi orientada pela necessidade de cobrir paradigmas metodológicos distintos, permitindo uma comparação fundamentada entre abordagens estatísticas clássicas e métodos baseados em aprendizado profundo.

Nesse sentido, foram selecionados a princípio, três modelos: o ARIMA, a Rede Neural Recorrente simples (RNN) e a Rede de Memória Longa de Curto Prazo (LSTM).

Os critérios para seleção foram: (1) **Progressão de complexidade**: ARIMA (modelo clássico, estatístico) → RNN (neural simples) → LSTM (neural com gates). Isto permite identificar em qual ponto a complexidade traz ganho real. (2) **Representatividade de paradigmas**: cobrindo abordagem estatística, neural simples e neural avançada. (3) **Adequação ao problema**: todos três são apropriados para séries temporais em IoT. Estes pontos serão descritos com referências e de maneira mais profunda abaixo.

Outras alternativas foram consideradas (XGBoost, Prophet) mas não incluídas porque: XGBoost não é recorrente (inadequado para séries longas); Prophet é baseado em tendência, não em detecção de anomalia.

Essa progressão é metodologicamente importante porque permite identificar em que ponto a complexidade adicional do modelo se traduz em ganho efetivo de desempenho para o problema específico de detecção de anomalias em redes LoRaWAN.

O ARIMA (*Autoregressive Integrated Moving Average*) é um modelo estatístico clássico amplamente utilizado para modelagem e previsão de séries temporais estacionárias (BOX et al., 2015). Sua formulação combina três componentes: um componente autorregres-

sivo, que modela a dependência linear entre o valor atual e valores passados da série; um componente de integração, que torna a série estacionária por meio de diferenciação; e um componente de média móvel, que modela a dependência em relação aos erros de previsão anteriores. A principal vantagem do ARIMA neste contexto é sua interpretabilidade: os parâmetros do modelo têm significado estatístico direto e o comportamento do modelo pode ser analisado formalmente. Por esse motivo, o ARIMA é utilizado neste trabalho como *baseline* estatístico, estabelecendo um patamar de referência para a avaliação dos modelos neurais subsequentes. Sua limitação principal reside na suposição de linearidade das dependências temporais, o que pode ser insuficiente para capturar padrões mais complexos presentes em séries de redes de sensores.

As Redes Neurais Recorrentes (RNNs) representam uma extensão natural das redes neurais artificiais para o domínio de sequências temporais (RUMELHART; HINTON; WILLIAMS, 1986). Diferentemente das redes neurais tradicionais, as RNNs mantêm um estado interno que é atualizado a cada instante de tempo, permitindo que informações de observações anteriores influenciem a previsão atual. Essa característica as torna adequadas para modelar dependências temporais sem a necessidade de especificação manual da ordem de defasagem, como ocorre no ARIMA. Neste trabalho, a RNN simples é utilizada como *baseline* neural, representando o paradigma recorrente em sua forma mais elementar e servindo de referência para a avaliação da LSTM.

A Rede de Memória Longa de Curto Prazo (LSTM, do inglês *Long Short-Term Memory*) é uma arquitetura recorrente desenvolvida especificamente para superar uma limitação fundamental das RNNs simples: o problema do desvanecimento do gradiente durante o treinamento (HOCHREITER; SCHMIDHUBER, 1997). Em sequências longas, as RNNs tendem a perder a capacidade de propagar informações relevantes de instantes distantes no tempo, o que limita sua eficácia na modelagem de dependências temporais de longo alcance. A LSTM contorna esse problema por meio de uma estrutura de células de memória controladas por portões de entrada, esquecimento e saída, que regulam seletivamente quais informações devem ser retidas, atualizadas ou descartadas ao longo da sequência. Essa capacidade é especialmente relevante no contexto de redes LoRaWAN, onde padrões anômalos podem se manifestar como desvios sutis acumulados ao longo de múltiplos quadros consecutivos, e não necessariamente como eventos isolados e abruptos. Por esse conjunto de razões, a LSTM é tratada neste trabalho como o modelo principal da abordagem proposta, sendo avaliada em comparação com os dois *baselines* anteriores.

A avaliação comparativa dos três modelos será conduzida com base em métricas de erro de regressão, notadamente o Erro Absoluto Médio (MAE) e a Raiz do Erro Quadrático Médio (RMSE), aplicadas sobre o conjunto de teste. Para a etapa de detecção de anomalias, o resíduo de cada modelo será submetido a um critério de limiarização, e o desempenho da detecção será avaliado por meio de métricas como precisão, revocação e taxa de falsos

positivos. Essa distinção entre a avaliação da qualidade de previsão e a avaliação da capacidade de detecção é metodologicamente necessária, pois um modelo pode apresentar bom desempenho de regressão sem necessariamente produzir resíduos bem calibrados para a identificação de anomalias.

2.3.3 Considerações sobre Inferência em Dispositivos de Borda

O treinamento e a avaliação dos modelos descritos neste trabalho são realizados em ambiente computacional convencional, com recursos de memória e processamento sem restrições relevantes. Essa decisão é adequada à fase atual do projeto, cujo objetivo central é investigar a eficácia das abordagens de detecção de anomalias antes de considerar questões de implantação.

No entanto, uma perspectiva relevante para a continuidade do trabalho envolve a possibilidade de execução do modelo treinado diretamente no gateway, eliminando a necessidade de transmissão dos dados brutos para uma camada externa de processamento. Esse paradigma, frequentemente referenciado na literatura como *TinyML* ou inferência em borda, consiste na adaptação de modelos de aprendizado de máquina para execução em microcontroladores com severas restrições de memória RAM, armazenamento em *flash* e consumo energético (RAY, 2022).

A viabilidade dessa portabilidade depende fundamentalmente da complexidade do modelo resultante após o treinamento. Modelos com menor número de parâmetros e estruturas mais simples, como RNNs com poucas unidades recorrentes ou versões reduzidas da LSTM, são candidatos mais adequados à conversão para formatos otimizados de inferência, como o TensorFlow Lite for Microcontrollers. Técnicas adicionais de compressão, como quantização de pesos e poda de conexões, podem reduzir ainda mais o custo computacional sem perda significativa de desempenho (JACOB et al., 2018).

Neste trabalho, a análise da complexidade dos modelos treinados será conduzida como critério complementar de avaliação, com o objetivo de fornecer subsídios para essa investigação futura. Contudo, a portabilidade efetiva para o gateway não constitui um requisito do projeto atual, sendo tratada como uma perspectiva de continuidade a ser explorada em trabalhos subsequentes, conforme discutido na Seção 7.3.

3 Método do trabalho

Este capítulo apresenta o método adotado na construção da proposta deste trabalho. A definição da solução foi conduzida a partir de três frentes complementares: levantamento bibliográfico, análise técnica das tecnologias envolvidas e organização da arquitetura do sistema. O objetivo desse processo foi estruturar uma proposta coerente para comunicação, segurança e análise de dados em um cenário de sistemas embarcados e Internet das Coisas.

Inicialmente, é apresentada a arquitetura de comunicação e infraestrutura do sistema, que estabelece o caminho de coleta, encaminhamento e armazenamento dos dados. Em seguida, são apresentadas as etapas relacionadas à segurança e ao uso de aprendizado de máquina, consideradas como partes integradas à solução proposta. Ao final, o capítulo organiza os elementos que orientam a implementação e a avaliação do sistema.

3.1 Procedimento de definição da arquitetura do sistema

A definição da arquitetura do sistema foi conduzida a partir da combinação entre levantamento bibliográfico, análise técnica das tecnologias envolvidas e separação em módulos da solução proposta. O objetivo dessa etapa foi estruturar uma arquitetura coerente com o problema estudado, considerando as restrições típicas de sistemas embarcados, a necessidade de comunicação em campo e a integração posterior com mecanismos de segurança e análise.

Como referência metodológica para essa definição, o trabalho adota princípios inspirados no modelo RM-ODP, utilizando diferentes pontos de vista para organizar a solução. Em termos funcionais, foram identificados os principais blocos responsáveis pela coleta, encaminhamento, recebimento e armazenamento das informações. Em termos de informação, foram considerados os dados de aplicação e os metadados necessários para rastreabilidade, segurança e análise posterior. Em termos de engenharia, foram observadas as separações entre o trecho embarcado da comunicação, o gateway e a infraestrutura de backend.

A partir dessa organização, a arquitetura foi estruturada de modo a separar claramente a comunicação em campo da camada de aplicação. Essa separação permite tratar, de forma independente, as restrições energéticas dos nós sensores, o papel intermediário do gateway e as funções de autenticação, persistência e análise associadas ao backend. Com isso, a definição arquitetural deixa de ser apenas uma escolha de componentes e passa a refletir a distribuição de responsabilidades entre as partes do sistema.

Além da decomposição dos componentes, o procedimento de definição da arquitetura

também considerou o fluxo das informações ao longo da solução. Para isso, foram analisadas as interações entre os nós de campo, o gateway e a infraestrutura de backend, buscando garantir que o sistema fosse capaz de receber, registrar e armazenar dados e metadados de forma compatível com as etapas posteriores do trabalho.

Por fim, a representação da interação entre os principais componentes foi organizada também por meio de diagramas, de modo a tornar mais clara a sequência de eventos envolvida na coleta, no encaminhamento e no armazenamento das mensagens. Essa modelagem auxilia tanto na compreensão da proposta quanto na transição entre a etapa metodológica e a etapa posterior de especificação da arquitetura.

3.1.1 Critérios adotados na decomposição arquitetural

A modularização da arquitetura foi orientada por três critérios principais. O primeiro foi a separação entre o trecho de comunicação em campo e a infraestrutura de aplicação, de modo a preservar as vantagens do uso de LoRaWAN no nível embarcado e da comunicação baseada em rede IP no nível de backend. O segundo foi a definição de responsabilidades distintas para coleta, encaminhamento, autenticação e persistência, evitando concentrar funções heterogêneas em um único componente. O terceiro foi a necessidade de manter a solução compatível com a incorporação posterior de mecanismos de criptografia leve e análise por aprendizado de máquina.

Esses critérios levaram à definição de uma arquitetura em blocos com papéis bem delimitados. No trecho embarcado, concentram-se as funções de aquisição e transmissão inicial dos dados. No ponto de transição, o gateway atua como elemento de integração entre a rede de campo e a infraestrutura de aplicação. No backend, concentram-se as funções de autenticação, persistência e preparação dos dados para uso posterior. Essa divisão favorece clareza arquitetural, reduz acoplamento entre responsabilidades e facilita a evolução incremental da solução.

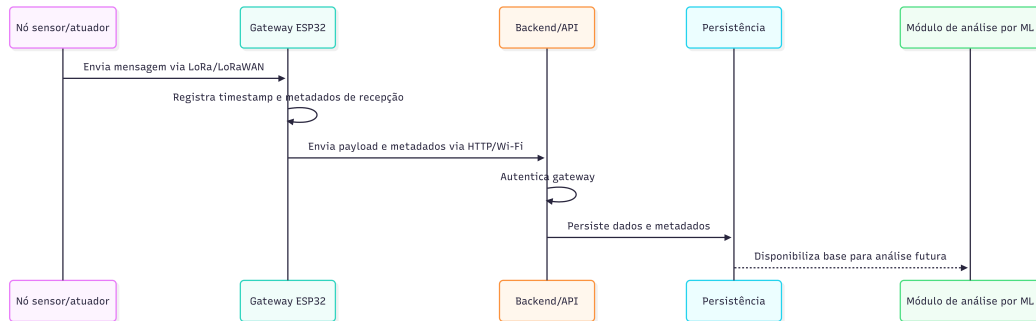
3.1.2 Modelagem das interações e do fluxo de dados

Para explicitar a dinâmica de funcionamento da proposta, o fluxo principal de interação entre os componentes foi modelado em termos de sequência de eventos. Essa modelagem considera a coleta dos dados no nó de campo, a transmissão pela rede LoRaWAN, a recepção e o enriquecimento da mensagem no gateway, o encaminhamento ao backend por Wi-Fi e HTTP, e o armazenamento final de dados e metadados na infraestrutura de persistência.

A utilização de um diagrama de sequência nessa etapa é importante porque permite representar, de forma mais direta, a ordem das interações entre os componentes e os pontos em que ocorre o tratamento das mensagens. Com isso, o fluxo deixa de ser apenas uma

descrição textual e passa a ser apresentado também como uma sequência organizada de ações entre entidades bem definidas.

Figura 8 – Diagrama de sequência UML do fluxo principal



Essa representação também ajuda a evidenciar que o sistema foi concebido para operar de forma orientada a evento. Em vez de depender de comunicação contínua, o fluxo é ativado pela geração e recepção das mensagens produzidas em campo, o que torna a proposta mais compatível com o tipo de aplicação e com as restrições energéticas associadas aos dispositivos embarcados.

3.2 Especificação de Requisitos e Revisão Bibliográfica

3.2.1 Materiais e Bases de Dados

Para viabilizar a etapa de modelagem, foi realizada uma pesquisa orientada à identificação de *datasets* abertos, confiáveis e suficientemente completos para sustentar análises, treinamento de modelos e comparação de resultados. A seleção dos conjuntos de dados foi guiada por três critérios principais: aderência ao contexto de LoRaWAN, disponibilidade de atributos úteis para engenharia de *features* e complementaridade entre cenários de operação normal e cenários com potencial presença de anomalias ou ataques.

A seleção de *datasets* públicos é uma escolha orientada por cronograma e recursos do projeto. Pois *Datasets* públicos permitem focar tempo no desenvolvimento de modelos e validação metodológica, em vez de depender da infraestrutura de captura. Além disso, dados coletados em operação real (não simulados) e já diversificados em ambiente real permitem comparação com outros trabalhos em literatura realizados. Por estas razões, foram identificados dois *datasets* complementares nas subseções seguintes e que serão utilizados no trabalho.

3.2.1.1 Dataset Tour Perret — base principal de modelagem

O *dataset* principal adotado neste trabalho é o *Tour Perret LoRaWAN Frames Dataset*, pertencente ao ecossistema CampusIoT. O repositório oficial lista o conjunto com 421.937 entradas de *frames* LoRaWAN, coletadas a partir de dispositivos instalados na Tour Perret, em Grenoble, França. A plataforma CampusIoT se apresenta como uma infraestrutura de ensino e pesquisa voltada à experimentação *in-vivo*, apoiada por uma rede LoRaWAN privada operando nas metrópoles de Grenoble e Valence ([CampusIoT, 2023a](#); [CampusIoT, 2023b](#)).

A principal característica que justifica a escolha desse *dataset* como base principal é sua natureza de operação real e contínua. Os dados foram produzidos por sensores em funcionamento efetivo, ao longo de um período extenso, sem intervenção artificial no tráfego. Essa característica implica diretamente na ausência de rótulos de ataque, o que, longe de ser uma limitação, define com precisão a abordagem metodológica adequada para este trabalho: a modelagem do comportamento normal da rede, a partir da qual desvios podem ser identificados como potencialmente anômalos. Em outras palavras, o Tour Perret é ideal para o treinamento de modelos de regressão temporal voltados à detecção de anomalias não supervisionada, justamente porque representa com fidelidade as condições que um sistema de monitoramento real encontraria em campo.

Além disso, o volume de entradas do *dataset* é suficiente para sustentar as etapas de treinamento, validação e teste com divisão temporal, sem comprometer a representatividade de cada partição. A diversidade de dispositivos e a continuidade temporal das observações também favorecem a construção de um modelo robusto ao comportamento variado de diferentes nós da rede.

3.2.1.2 Análise Exploratória do Dataset Tour Perret

O *dataset* Tour Perret foi submetido a uma análise exploratória preliminar com o objetivo de caracterizar sua estrutura, verificar a qualidade dos dados disponíveis e identificar os atributos mais relevantes para a etapa de modelagem. Essa análise constitui parte integrante do método adotado, fornecendo evidências concretas que embasam as decisões de seleção de variáveis descritas na Seção 3.2.2.

O conjunto de dados é composto por 421.937 entradas de *frames* LoRaWAN, distribuídas entre três cenários identificados pelo campo `applicationName`: o cenário ELSYS_EMS, predominante com 412.380 registros (97,7% do total), o cenário WYRES_BASE, com 9.526 registros, e um terceiro identificador com 31 entradas. Essa concentração em um único cenário de aplicação é metodologicamente favorável, pois permite que a modelagem do comportamento normal seja construída sobre uma base homogênea, reduzindo a influência de padrões heterogêneos entre aplicações distintas.

A verificação de cobertura dos campos revelou alta qualidade dos dados para todos os atributos relevantes ao projeto, conforme apresentado na Tabela 1. Os campos `timestamp`, `devEUI` e `rxInfo` apresentaram preenchimento de 100%, enquanto `fCnt`, `fPort` e `_redundancy` atingiram 99,99%, e o campo `object`, que contém o *payload* decodificado, registrou 99,98% de preenchimento. Essa consistência indica que o *dataset* é adequado para o treinamento de modelos de regressão temporal sem necessidade de estratégias complexas de imputação de dados ausentes.

Tabela 1 – Cobertura dos campos relevantes no dataset Tour Perret

Atributo	Campo original	Preenchimento (%)
timestamp	<code>_timestamp</code>	100,00
device_id	<code>devEUI</code>	100,00
gateway_info	<code>rxInfo</code>	100,00
fCnt	<code>fCnt</code>	99,99
freq_dr	<code>txInfo</code>	100,00
fPort	<code>fPort</code>	99,99
payload	<code>object</code>	99,98
redundância	<code>_redundancy</code>	99,99

Fonte: elaborado pelos autores.

A análise de diversidade identificou 5 dispositivos únicos no subconjunto analisado, com distribuição heterogênea de volume de mensagens entre eles. A variabilidade de RSSI por dispositivo foi calculada e revelou perfis distintos de enlace para cada nó da rede, conforme apresentado na Tabela 2. O dispositivo `42fb8334...` apresentou o maior desvio padrão de RSSI (11,30 dBm), indicando condições de enlace mais instáveis, enquanto o dispositivo `73d89239...` exibiu o menor desvio (5,83 dBm), sugerindo um enlace mais estável ao longo do tempo. As médias de RSSI variaram entre $-97,38$ dBm e $-107,76$ dBm entre os dispositivos, confirmando que o perfil de sinal é característico de cada nó e potencialmente utilizável como base para a modelagem de comportamento esperado individualizado.

Tabela 2 – Variabilidade de RSSI por dispositivo no dataset Tour Perret

devEUI (parcial)	Média (dBm)	Desvio padrão	Mín	Máx
2951cd7c...	$-106,43$	6,76	-127	-59
42fb8334...	$-97,38$	11,30	-122	-74
441a1ed9...	$-103,45$	9,01	-127	-76
73d89239...	$-107,76$	5,83	-126	-68
73fe90c2...	$-103,64$	10,02	-130	-55

Fonte: elaborado pelos autores.

As *features* numéricas RSSI, SNR, frequência de operação e *data rate* foram extraídas com sucesso das estruturas aninhadas dos campos `rxInfo` e `txInfo`, confirmando

a viabilidade de utilização desses atributos nos modelos de regressão. O fato de que essas variáveis estão disponíveis de forma consistente em praticamente todas as entradas do *dataset* reforça sua adequação como candidatas principais à seleção final de variáveis, discutida na Seção 3.2.2.

Em conjunto, os resultados da análise exploratória confirmam que o *dataset* Tour Perret apresenta qualidade, volume e riqueza de atributos suficientes para sustentar a etapa de modelagem proposta neste trabalho, tanto em termos de dados de treinamento quanto de variáveis candidatas para a construção dos modelos de regressão temporal.

3.2.1.3 Dataset GUARD — papel complementar e de referência

O segundo *dataset* considerado neste trabalho é o *LoRaWAN Network Attacks*, associado ao projeto GUARD. De acordo com sua descrição oficial, esse conjunto contém um *dump* de mensagens observadas em uma rede LoRaWAN durante a validação do projeto, incluindo tráfego legítimo proveniente de dispositivos reais instalados em ônibus urbanos, além de ataques artificialmente gerados por replicação ou alteração de mensagens (GUARD Project, 2023).

3.2.1.3.1 Caracterização dos Ataques em GUARD

Uma análise exploratória do *dataset* GUARD foi conduzida para caracterizar a natureza dos ataques contidos. Embora o *dataset* não forneça rótulos explícitos (coluna binária de ataque/normal), contém indicadores comportamentais estruturados que permitem identificação confiável de eventos anômalos.

A análise focou nas colunas **type** e **error**, que registram eventos durante processamento de mensagens LoRaWAN. Foram identificados dois indicadores principais:

1. **UPLINK_FCNT_RETRANSMISSION**: Frame counter não incrementou sequencialmente, indicando retransmissão de mensagem anterior (ataque de replicação).
2. **UPLINK_FCNT_RESET**: Frame counter sofreu reset inesperado, indicando contador adulterado ou dispositivo comprometido (ataque de spoofing).

A filtragem sistemática identificou **6.799 registros** com um ou mais destes indicadores, representando eventos anômalos confirmados. Esta quantidade significativa fornece volume adequado para validação de capacidade de detecção.

3.2.1.3.2 Estratégias de Validação com GUARD

Diferentemente do Tour Perret, o *dataset* GUARD contém indicadores de ataque explicitamente identificáveis através de suas colunas **type** e **error**. No contexto deste

trabalho, contudo, ele exerce um papel complementar e não central, sendo utilizado em duas estratégias de validação distintas.

Estratégia 1: Teste de Generalização Após treinar o modelo com Tour Perret (aprendendo padrões de comportamento normal), aplicar este modelo aos dados de GUARD para verificar se consegue identificar desvios associados a ataques conhecidos. Esta abordagem responde à questão: “*O modelo consegue detectar anomalias mesmo quando confrontado com contexto diferente daquele em que foi treinado?*”

Importa notar que esta validação possui uma limitação intrínseca: Tour Perret (sensores em Grenoble) e GUARD (ônibus urbanos) representam contextos distintos. Portanto, quando o modelo detecta desvios em GUARD, isto pode resultar tanto de (i) identificação de ataque quanto de (ii) mudança de contexto ambiental. Assim, esta abordagem valida *sensibilidade* do modelo a anomalias, mas não isoladamente a capacidade de distinguir ataques de mudanças contextuais.

Estratégia 2: Validação Supervisionada (Se Viável) Se análise mais profunda de GUARD indicar viabilidade, uma segunda estratégia envolve treinar um classificador supervisionado (Normal vs Ataque) exclusivamente dentro de GUARD, usando os 6.799 registros anômalos como “ataque” e registros restantes como “normal”. Validação cruzada dentro de GUARD forneceria comparação direta com a abordagem não supervisionada de Tour Perret.

Esta estratégia enfrenta dois desafios práticos reconhecidos:

1. **Engenharia de features:** Campos estruturados de GUARD (`time`, `tx_info`, `rx_info`, `data`) requerem transformação significativa em features numéricas compatíveis com modelos de aprendizado de máquina, consumindo tempo de desenvolvimento.
2. **Desbalanceamento de classes:** O *dataset* GUARD possui proporção desigual entre registros normais e registros de ataque. Treinar modelos supervisionados com desbalanceamento severo introduz viés. Técnicas de balanceamento (oversampling, undersampling, pesos de classe) seriam necessárias.

Por estas razões, a abordagem supervisionada é considerada secundária e explorada apenas se cronograma permitir. Do contrário, será considerada uma perspectiva de continuidade, como futuros passos.

Objetivos Distintos, Insights Complementares As duas estratégias respondem a questões metodologicamente diferentes:

1. **Tour Perret (Não Supervisionado)**: “O modelo consegue aprender padrões normais e detectar desvios significativos?” Fornece validação da abordagem de regressão temporal em dados reais de operação normal.
2. **GUARD - Generalização**: “O modelo generaliza para novo contexto?” Fornece indicação de robustez do modelo a variações ambientais.
3. **GUARD - Supervisionado** (se viável): “O modelo consegue classificar quando dispõe de rótulos explícitos?” Fornece comparação com baseline supervisionado.

A combinação destas estratégias enriquece a análise sem exigir que o modelo principal seja treinado exclusivamente com dados rotulados, mantendo o cronograma viável.

3.2.2 Seleção de Variáveis

A definição das variáveis de entrada dos modelos de regressão é uma etapa metodológica central neste trabalho, pois determina diretamente quais aspectos do comportamento da rede serão modelados e, conseqüentemente, quais tipos de anomalia o sistema será capaz de detectar. A seleção foi conduzida a partir de dois critérios principais: relevância para a caracterização do comportamento temporal da rede LoRaWAN e disponibilidade consistente nos *datasets* selecionados.

O *dataset* Tour Perret disponibiliza um conjunto de atributos associados a cada *frame* recebido, entre os quais se destacam como candidatos naturais à modelagem temporal as seguintes variáveis: a intensidade do sinal recebido (RSSI), a relação sinal-ruído (SNR), o contador de *frames* (FCnt), o intervalo de tempo entre mensagens consecutivas de um mesmo dispositivo (*inter-arrival time*) e a taxa de dados utilizada na transmissão (*data rate*). Cada uma dessas variáveis apresenta propriedades distintas do ponto de vista da detecção de anomalias.

O RSSI e o SNR refletem as condições do enlace de rádio e tendem a variar de forma gradual em condições normais de operação. Variações abruptas nesses indicadores podem sinalizar interferência deliberada, mudança de posição do dispositivo ou substituição fraudulenta de um nó — padrões associados a ataques de *jamming* e *spoofing*, respectivamente. O FCnt, por sua vez, deve apresentar incremento monotônico e regular ao longo do tempo para um dispositivo legítimo. Quebras nesse padrão, como saltos inesperados ou regressões no contador, são características típicas de ataques de *replay*, nos quais mensagens anteriores são retransmitidas por um adversário. O *inter-arrival time* captura o ritmo de transmissão do dispositivo e é especialmente sensível a comportamentos de *flooding* ou a tentativas de esgotamento do canal (HESSEL; ALMON; HOLLICK, 2023). O

data rate, embora ajustado dinamicamente pelo mecanismo ADR, também pode apresentar variações anômalas em situações de interferência ou manipulação da comunicação.

A seleção final das variáveis a serem utilizadas nos modelos será definida com base na análise exploratória dos dados do Tour Perret descrita na Seção 3.2.1.2, considerando a estabilidade temporal de cada atributo, a presença de valores ausentes e a correlação entre variáveis candidatas. O objetivo é identificar um subconjunto reduzido — preferencialmente de até três variáveis — que concentre o maior poder explicativo para a modelagem do comportamento normal da rede, mantendo a complexidade do modelo compatível com uma eventual análise de portabilidade futura para dispositivos de borda.

3.2.3 Procedimentos de Treinamento e Avaliação

O treinamento dos modelos de regressão será realizado em ambiente computacional convencional, sem restrições relevantes de memória ou processamento. Essa decisão é adequada à fase atual do projeto, cujo objetivo é avaliar a eficácia das abordagens de detecção de anomalias antes de qualquer consideração sobre implantação em dispositivos restritos.

3.2.3.1 Preparação e Divisão dos Dados

Antes do treinamento, os dados do Tour Perret serão submetidos a uma etapa de preparação que inclui harmonização de *timestamps*, tratamento de valores ausentes, normalização das variáveis selecionadas e organização das sequências por dispositivo. A divisão dos dados entre conjuntos de treinamento, validação e teste seguirá uma lógica de divisão temporal (*temporal split*), na qual as observações mais antigas compõem o conjunto de treinamento e as mais recentes compõem os conjuntos de validação e teste. Essa abordagem é metodologicamente necessária para séries temporais, pois preserva a ordem cronológica dos dados e evita o vazamento de informação futura para o treinamento (*data leakage*), problema que ocorreria em divisões aleatórias (BERGMEIR; BENÍTEZ, 2012; HYNDMAN; ATHANASOPOULOS, 2021).

3.2.3.2 Treinamento dos Modelos

Os três modelos descritos na Seção 2.3.2 — ARIMA, RNN e LSTM — serão treinados sobre o conjunto de treinamento, exclusivamente com dados que representam a operação normal da rede. Para o ARIMA, a ordem dos componentes autorregressivo, de integração e de média móvel será determinada por meio de análise das funções de autocorrelação e autocorrelação parcial da série (BOX et al., 2015). Para a RNN e a LSTM, a arquitetura das redes será definida de forma a manter o número de parâmetros em um nível compatível com a complexidade do problema, favorecendo a generalização e a eventual análise de portabilidade.

3.2.3.3 Avaliação da Qualidade de Regressão

A qualidade de previsão de cada modelo será avaliada sobre o conjunto de teste por meio do Erro Absoluto Médio (MAE) e da Raiz do Erro Quadrático Médio (RMSE). Essas métricas quantificam a magnitude média dos erros de previsão, sendo o RMSE mais sensível a erros grandes e pontuais, e o MAE mais robusto a *outliers* (HYNDMAN; ATHANASOPOULOS, 2021).

Formalmente, o Erro Absoluto Médio é calculado como:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

e a Raiz do Erro Quadrático Médio é:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

em que n é o número de observações no conjunto de teste, y_i é o valor observado e $\hat{y}_i$ é o valor previsto pelo modelo no instante i .

O MAE mede o erro médio em termos absolutos, oferecendo interpretação direta na unidade da variável. O RMSE, ao elevar os erros ao quadrado antes da média, penaliza mais fortemente erros grandes, tornando-o mais sensível a *outliers*. Assim, o MAE é mais robusto a anomalias pontuais, enquanto o RMSE fornece indicação de quão grave são os erros de previsão quando ocorrem.

3.2.3.4 Avaliação da Capacidade de Detecção de Anomalias

A etapa de detecção de anomalias é conduzida de forma independente da avaliação de regressão, pois um modelo com bom desempenho de previsão não produz necessariamente resíduos bem calibrados para a identificação de desvios (CHALAPATHY; CHAWLA, 2019). O procedimento consiste em calcular o resíduo entre o valor previsto e o valor observado para cada instante do conjunto de teste e aplicar um critério de limiarização para classificar cada observação como normal ou potencialmente anômala. O limiar será definido com base na distribuição dos resíduos observados no conjunto de validação, adotando como referência um múltiplo do desvio padrão dos erros de treinamento.

Para a avaliação da detecção, serão utilizadas métricas de classificação binária, incluindo precisão, revocação e taxa de falsos positivos. No contexto de segurança em redes IoT, a revocação — isto é, a capacidade do modelo de identificar corretamente eventos anômalos — é a métrica de maior relevância operacional, pois falhas na detecção de ataques reais têm consequências mais graves do que a geração de alarmes falsos (HESSEL; ALMON; HOLLICK, 2023). A taxa de falsos positivos, por sua vez, é igualmente importante do

ponto de vista da operabilidade do sistema, uma vez que um número elevado de alarmes incorretos pode comprometer a confiança e a utilidade prática da solução.

3.2.4 Procedimentos de Treinamento e Avaliação

A implementação da abordagem de *Machine Learning* foi concebida a partir da hipótese de que anomalias em redes LoRaWAN podem ser identificadas por meio de desvios em padrões temporais, atributos de enlace e características do tráfego observado. Com base nisso, a estratégia de modelagem foi estruturada em etapas sucessivas: inspeção e padronização dos dados, seleção e engenharia de atributos, treinamento de modelos, comparação entre abordagens e análise dos resultados.

Na etapa de preparação dos dados, torna-se necessário harmonizar *timestamps*, normalizar identificadores de dispositivos e *gateways*, tratar campos ausentes e organizar atributos comuns entre datasets distintos. Em seguida, são construídas variáveis de entrada que representem tanto o estado instantâneo do *frame* quanto seu contexto temporal. Entre os atributos mais relevantes para essa análise, destacam-se métricas como RSSI, SNR, frequência de operação, *data rate*, presença de redundância de recepção, progressão de *frame counters*, intervalo entre mensagens e padrões de repetição de *payload* ou comportamento por dispositivo.

A modelagem pode então ser conduzida por duas frentes complementares. A primeira envolve métodos voltados à detecção de anomalias com pouca ou nenhuma dependência de rótulos completos, o que é particularmente útil em cenários reais de IoT. A segunda envolve, quando houver suporte suficiente dos dados, experimentos comparativos com métodos supervisionados ou semi-supervisionados, sobretudo nos casos em que o dataset apresenta descrição de tráfego manipulado ou cenários de ataque. Essa combinação é apropriada porque permite comparar não apenas acurácia bruta, mas também robustez metodológica diante de diferentes qualidades de rotulação.

Além disso, a estratégia adotada considera que a utilidade dos modelos não deve ser avaliada apenas em função do desempenho estatístico, mas também em relação à sua compatibilidade com execução futura em dispositivos restritos. Por esse motivo, o desenvolvimento da modelagem já considera a complexidade dos métodos, o número de atributos utilizados e o custo potencial de inferência, buscando favorecer posteriormente sua adaptação para contextos de borda.

3.2.5 Biblioteca Ascon

Para a implementação das primitivas criptográficas no microcontrolador, optou-se pela utilização da biblioteca de código aberto *ascon-suite*, mantida por [Weatherley \(2023\)](#). A escolha desta implementação específica, em detrimento do repositório de referência

matemático original (*ascon-c*), justifica-se por fatores arquiteturais estritamente alinhados aos objetivos de prototipagem em sistemas embarcados.

Primeiramente, a *ascon-suite* foi estruturada com foco em plataformas com restrição de recursos, fornecendo rotinas de *backend* altamente otimizadas em linguagem *Assembly*, podendo ser usada diretamente na IDE do Arduino como uma biblioteca, facilitando o desenvolvimento na placa ESP32.

Em segundo lugar, a biblioteca expõe uma Interface de Programação de Aplicações (API) de alto nível que suporta processamento de dados de forma incremental. Essa característica permite a criptografia de fluxos e *payloads* em pequenos fragmentos, o que é um recurso imperativo para garantir que o microcontrolador não esgote sua limitada memória RAM ao tentar alocar mensagens inteiras de uma só vez antes da transmissão via rádio.

Por fim, o rigor técnico da biblioteca possui forte validação científica. Os códigos otimizados em *Assembly* e C que deram origem à *ascon-suite* foram ativamente empregados como base metodológica para a extração de métricas de desempenho em microcontroladores durante o processo de padronização do NIST, tendo seus resultados oficialmente documentados na seção 4.1.3 do relatório governamental NIST IR 8454 ([TURAN et al., 2023](#)).

3.3 Atividades propostas para desenvolvimento e validação

As atividades propostas nesta etapa do trabalho foram organizadas de modo a permitir a evolução progressiva da solução, partindo da infraestrutura básica de comunicação e avançando, posteriormente, para a incorporação das camadas de segurança e análise. Essa organização busca reduzir a complexidade da integração inicial e tornar mais clara a relação entre os diferentes módulos do sistema ao longo do desenvolvimento do projeto.

3.3.1 Integração progressiva da infraestrutura

A primeira frente de atividade considera a integração da infraestrutura básica de comunicação, envolvendo os nós de campo, o gateway e a camada de backend. Nessa etapa, o foco está no estabelecimento do fluxo principal do sistema, de modo que os dados produzidos em campo possam ser recebidos, encaminhados, autenticados e armazenados de forma consistente.

Em seguida, a proposta considera a consolidação da persistência dos dados e metadados gerados ao longo da comunicação. Essa base é necessária não apenas para o funcionamento da solução, mas também para dar suporte às etapas seguintes do trabalho. Após essa consolidação, passam a ser incorporadas as atividades relacionadas à proteção

criptográfica dos dados e à exploração analítica das informações persistidas.

3.3.2 Atividades propostas de validação

As atividades de validação previstas no trabalho acompanham a evolução da integração da solução. Inicialmente, será observada a consistência do fluxo de comunicação entre os dispositivos de campo, o gateway e o backend. Em seguida, serão consideradas atividades voltadas ao registro correto de metadados, à persistência das informações e ao comportamento do sistema em situações de falha temporária de conectividade.

Em etapas posteriores, a validação passa a incluir a incorporação da camada criptográfica baseada em Ascon e o uso da base persistida em análises por aprendizado de máquina. Com isso, a avaliação deixa de incidir apenas sobre a comunicação básica e passa a contemplar também o comportamento integrado da proposta, reunindo infraestrutura, segurança e análise de dados.

3.3.3 Relação com o cronograma do projeto

A organização dessas atividades em etapas também tem a função de alinhar a evolução técnica da solução ao cronograma do projeto. Em um primeiro momento, concentram-se os esforços na definição e integração da infraestrutura básica. Em seguida, a continuidade do trabalho contempla a inserção progressiva das demais frentes, de modo que a arquitetura possa ser avaliada em níveis crescentes de complexidade.

Essa organização permite que cada etapa produza uma base para a etapa seguinte, reduzindo retrabalho e favorecendo a coerência do desenvolvimento como um todo. Dessa forma, o cronograma deixa de ser apenas uma divisão temporal de tarefas e passa a refletir a lógica de construção incremental da proposta.

3.4 Testes e Avaliação de Desempenho (*Benchmarking*)

3.4.1 *Known Answer Tests* (KAT)

A primeira etapa será a validação através dos *Known Answer Tests* (KAT), que são testes disponibilizados pelo próprio grupo que desenvolveu o Ascon. Eles fornecem parâmetros de entrada específicos que devem resultar em saídas exatas e já esperadas. Esses arquivos podem ser encontrados no repositório oficial *ascon-c* ([DOBRAUNIG et al., 2023b](#)).

Para validar a biblioteca de forma puramente matemática, os testes serão rodados exaustivamente em um computador desktop. Depois, para garantir que não haja nenhuma

anomalia de compilação ou diferença causada pela arquitetura de hardware, testes adicionais serão executados diretamente na placa, mas com um volume menor de vetores.

Como garantia extra, antes de iniciar a coleta de desempenho, foram adicionados testes estáticos no código que rodam a cada inicialização do sistema para verificar a consistência. Ao avaliar as funções da biblioteca *ascon-suite* dessa maneira e comparar com os resultados oficiais, conseguimos comprovar de forma prática se a implementação está rodando corretamente no ambiente do projeto.

3.4.2 Validação dos Dados Associados (AEAD)

Outro teste a ser realizado é a validação da autenticidade do modo AEAD e a sua capacidade de rejeição de anomalias. Para avaliar a resiliência do sistema contra um ataque de *Man-in-the-middle* (MitM), no qual um interceptador tenta forjar ou corromper a mensagem em trânsito, a metodologia empregará a técnica de injeção intencional de falhas (*bit-flipping*).

Após a geração de um pacote válido, o firmware de teste irá alterar intencionalmente bits específicos antes de repassar a mensagem para a função de decifragem. Essa adulteração será testada em três frentes distintas:

- **No texto cifrado (*Ciphertext*):** simulando um atacante que tenta alterar o valor do *payload* do sensor.
- **Nos Dados Associados (AD):** alterando o cabeçalho do protocolo LoRaWAN, simulando uma tentativa de modificar metadados de roteamento ou de identificação do nó.
- **Na Tag de Autenticação:** simulando um cenário onde a mensagem é modificada e o atacante anexa uma assinatura forjada ou corrompida.

O resultado esperado em todos os cenários de adulteração é que a função de decifragem do algoritmo reconheça a quebra de integridade e falhe imediatamente, retornando um código de erro e garantindo que nenhum fragmento do texto claro modificado seja entregue para a camada de aplicação do microcontrolador.

3.4.3 Vazamento de Memória e Ataques de Canal Colateral

A avaliação de desempenho não se limitará apenas à validação funcional, englobando também o consumo de memória (RAM) e tempo de execução. Para a análise de memória, a metodologia empregará as funções nativas do *FreeRTOS* no ESP32 para capturar o pico máximo de uso de memória dinâmica e estática durante a execução da cifra. O uso do *Heap*

será monitorado por meio das funções `esp_get_free_heap_size()` antes e depois da execução, e `esp_get_minimum_free_heap_size()` para registrar a alocação máxima atingida. Adicionalmente, será medida a profundidade máxima alcançada na Pilha (*Stack*) da tarefa responsável pela criptografia através da função `uxTaskGetStackHighWaterMark()`, para verificar o espaço de estado interno ocupado pelo algoritmo.

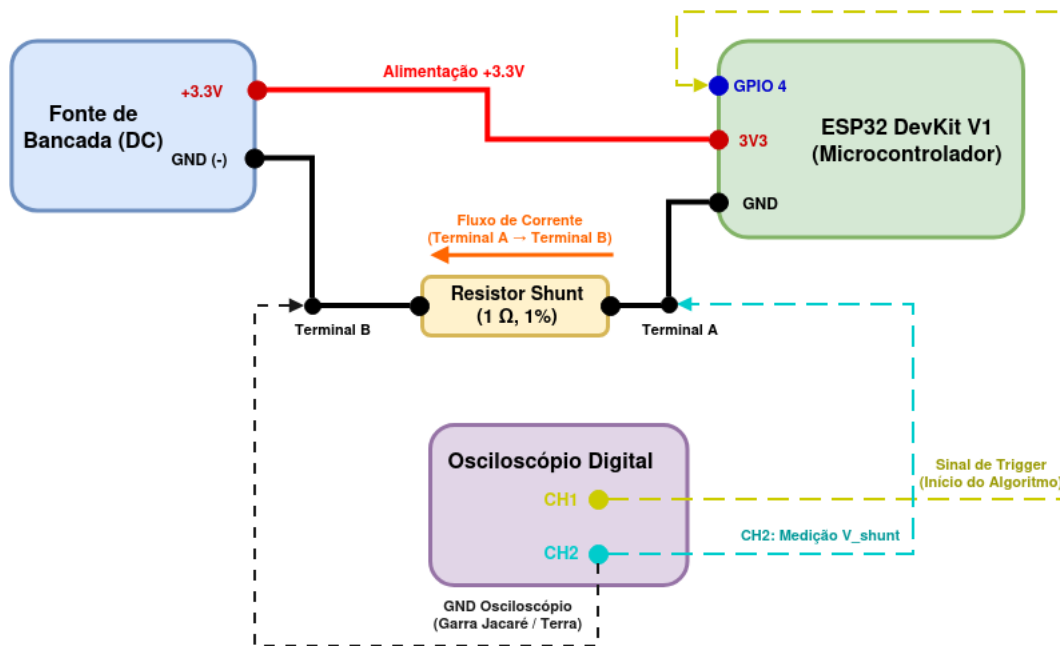
Outro aspecto avaliado será o espaço ocupado de memória. O procedimento consistirá em compilar o firmware e analisar o uso de memória Flash, comparando os resultados do compilador com e sem a inclusão da biblioteca no projeto.

Quanto ao tempo de execução, a metodologia buscará não apenas medir a eficiência, mas também validar a resistência das implementações contra ataques de canal colateral baseados em tempo (*Timing Side-Channel Attacks*). A medição extrairá diretamente a quantidade de ciclos de *clock* processados, lendo o registrador interno da CPU do ESP32 (através da instrução `esp_cpu_get_cycle_count()`). Para validar a mitigação de ataques de canal colateral, serão realizados testes repetitivos enviando cargas de dados (*payloads*) com conteúdos e chaves completamente distintos, mas de tamanhos idênticos. O resultado esperado é que a variância nos ciclos de *clock* consumidos seja estatisticamente nula, provando que o algoritmo executa em tempo constante para um determinado comprimento de dados, impedindo que um atacante deduza fragmentos da chave ou da mensagem com base no tempo de processamento do nó sensor.

3.4.4 Avaliação de Consumo Energético

A medição do consumo de energia será realizada através da técnica de monitoramento da queda de tensão em um resistor de precisão (*shunt*) de 1Ω conectado em série com a alimentação de 3,3V do microcontrolador (Hubble, 2024). As capturas serão feitas com um osciloscópio digital, utilizando um pino GPIO como sinal de gatilho (*trigger*) para delimitar com precisão o intervalo de tempo exato da execução das funções criptográficas.

Considerando que o módulo *ESP32 DevKitV1* possui componentes periféricos que consomem corrente de forma constante, como o LED de operação, o regulador de tensão LDO e o conversor USB-Serial, a análise não focará no consumo absoluto da placa. Em vez disso, será aplicada a metodologia de Diferença de Consumo (Δ), na qual subtrai-se a corrente de repouso (*baseline*) do sistema ocioso do valor obtido durante o processamento do algoritmo.

Figura 9 – Configuração de medição de corrente via resistor *shunt* e osciloscópio

Fonte: Elaborado pelos autores (2026) com base em (Hubble, 2024)

4 Especificação de Requisitos

4.1 Especificação da arquitetura do sistema

A arquitetura especificada neste trabalho define a organização técnica da solução proposta para comunicação, encaminhamento e persistência dos dados produzidos na rede de campo. Sua função é estruturar, de forma coerente, os componentes responsáveis pela coleta das informações, pela transição entre a comunicação embarcada e a infraestrutura de aplicação, e pelo armazenamento dos dados necessários ao funcionamento do sistema e às etapas posteriores de segurança e análise.

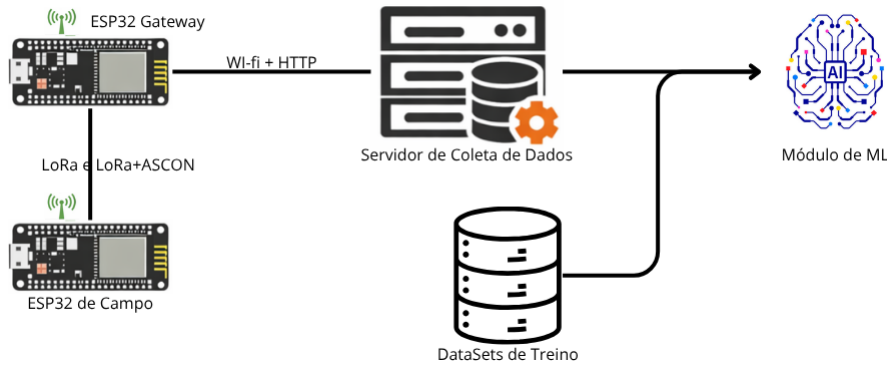
A solução foi organizada em quatro blocos principais: nós sensores e atuadores, gateway, backend/API e persistência. No trecho de campo, os nós realizam a coleta dos dados do ambiente e transmitem mensagens curtas por meio da arquitetura LoRaWAN, atendendo aos requisitos de operação em dispositivos com restrição energética e comunicação de baixo consumo. O gateway atua como elemento intermediário entre essa rede e a infraestrutura de aplicação, concentrando as funções de recepção, registro de metadados e encaminhamento das mensagens ao backend, em conformidade com os requisitos RF3 e RF6.

A partir do gateway, a comunicação passa a utilizar Wi-Fi e requisições HTTP, permitindo integrar a rede embarcada a uma API autenticada implantada em infraestrutura de nuvem e a um mecanismo centralizado de persistência. Essa organização busca manter a camada de aplicação disponível de forma contínua para o recebimento dos dados encaminhados pelo gateway, ao mesmo tempo em que facilita o armazenamento e a posterior utilização das informações coletadas.

Além dos dados de aplicação, como temperatura e luminosidade, a arquitetura prevê a persistência de metadados relevantes para o acompanhamento da comunicação, incluindo informações de origem, sequência, recepção e qualidade do enlace. Essa escolha atende aos requisitos RF5 e RF7, além de contribuir para a rastreabilidade da operação, conforme estabelecido em RNF6. A mesma base persistida também prepara a infraestrutura para a incorporação posterior de mecanismos de criptografia leve e de análise por aprendizado de máquina, em conformidade com RNF7 e RNF8.

A Figura 10 apresenta a arquitetura técnica especificada para o sistema. A partir dessa organização, as subseções seguintes detalham os requisitos funcionais e não funcionais da infraestrutura proposta, que fundamentam e justificam as decisões arquiteturais adotadas.

Figura 10 – Arquitetura especificada do sistema proposto.



4.1.1 Requisitos funcionais da infraestrutura proposta

Os requisitos funcionais da infraestrutura proposta descrevem os comportamentos esperados da arquitetura do sistema, considerando a coleta, o encaminhamento, a autenticação e a persistência das informações produzidas na rede de campo.

- RF1. O sistema deve permitir que os nós de campo colem e transmitam dados de aplicação, incluindo, no protótipo previsto, informações de temperatura e luminosidade.
- RF2. Cada mensagem transmitida pelos nós deve conter, além dos dados de aplicação, informações mínimas de identificação e acompanhamento, incluindo identificador do dispositivo e contador de mensagens.
- RF3. O gateway deve receber as mensagens provenientes da rede de campo, registrar os metadados de recepção e encaminhar essas informações ao backend por meio de rede Wi-Fi e requisições HTTP.
- RF4. O backend deve autenticar as requisições enviadas pelo gateway, restringindo o recebimento de dados a dispositivos previamente autorizados.
- RF5. O sistema deve persistir os dados de aplicação recebidos dos nós juntamente com os metadados associados à comunicação.
- RF6. A infraestrutura deve permitir armazenamento temporário das mensagens no gateway em situações de indisponibilidade da conexão com o backend, com possibilidade de reenvio posterior.
- RF7. Os dados persistidos devem permanecer disponíveis para uso posterior pelas camadas de segurança e de análise por aprendizado de máquina.

4.1.2 Requisitos não funcionais da infraestrutura proposta

Os requisitos não funcionais da infraestrutura proposta descrevem propriedades de operação e restrições que devem ser observadas para que a solução seja compatível com o contexto de sistemas embarcados e Internet das Coisas. Para sua elicitación e organização, adotou-se como referência a norma ISO/IEC 25010, considerando principalmente características relacionadas à eficiência de desempenho, confiabilidade, compatibilidade, manutenibilidade e segurança.

- RNF1. A infraestrutura deve ser compatível com a operação de nós de campo com restrições de energia, priorizando baixo consumo no trecho embarcado da comunicação.
- RNF2. A comunicação do sistema deve ser orientada a evento, evitando transmissões contínuas desnecessárias entre os componentes.
- RNF3. A arquitetura deve manter separação clara entre a rede de campo, baseada em LoRaWAN, e a infraestrutura de aplicação, baseada em Wi-Fi e HTTP.
- RNF4. O protótipo deve preservar simplicidade de integração entre hardware, comunicação e backend, de modo a manter viável sua montagem e evolução no contexto do projeto.
- RNF5. A infraestrutura deve apresentar robustez básica a falhas temporárias de conectividade entre gateway e backend.
- RNF6. O sistema deve garantir rastreabilidade das mensagens recebidas, por meio do registro de dados e metadados que permitam identificar origem, sequência, instante de recepção e condições básicas do enlace.
- RNF7. A arquitetura deve ser compatível com a incorporação posterior de mecanismos de criptografia leve sem exigir reformulação completa da infraestrutura de comunicação.
- RNF8. A infraestrutura deve ser compatível com o uso posterior de técnicas de aprendizado de máquina aplicadas à análise do comportamento da rede e à detecção de anomalias.

4.2 Requisitos do Sistema de Detecção de Anomalias

A incorporação de um modelo de aprendizado de máquina à arquitetura proposta introduz requisitos específicos, distintos dos requisitos de infraestrutura definidos na Seção 4.1. O modelo será treinado e avaliado em ambiente computacional convencional, utilizando os dados históricos do *dataset* Tour Perret. A viabilidade de portabilidade da fase de inferência para o gateway é tratada como perspectiva de continuidade, conforme discutido na Seção 7.3.

4.2.1 Requisitos Funcionais

- RF1. O sistema deve receber como entrada sequências temporais das variáveis selecionadas, organizadas por dispositivo e ordenadas cronologicamente.
- RF2. O sistema deve produzir previsões do comportamento esperado de cada variável monitorada e calcular o resíduo entre valor previsto e valor observado para cada instante de tempo.
- RF3. O sistema deve classificar cada observação como normal ou potencialmente anômala, com base em um critério de limiarização definido a partir do conjunto de validação.
- RF4. O sistema deve permitir comparação quantitativa entre as três abordagens de modelagem (ARIMA, RNN, LSTM) através de métricas padronizadas de qualidade de previsão e detecção.

4.2.2 Requisitos Não Funcionais

- RNF1. O treinamento deve ser realizado exclusivamente sobre dados que representem operação normal da rede, sem contaminação por observações potencialmente anômalas.
- RNF2. O processo de treinamento e avaliação deve ser reproduzível, com registro das sementes de inicialização, hiperparâmetros e critérios de seleção de ordem utilizados em cada modelo.

Essa orientação fortalece a coerência do trabalho como um todo. Em vez de separar artificialmente a fase de modelagem da fase de implantação, o desenvolvimento já considera que a utilidade prática dos modelos depende de um equilíbrio entre eficácia de detecção e viabilidade de execução em dispositivos restritos. Assim, a implementação proposta dialoga diretamente com a etapa posterior de avaliação energética e computacional.

Sob essa perspectiva, a comparação entre modelos não deverá considerar apenas métricas tradicionais de desempenho, mas também aspectos como simplicidade estrutural, custo computacional, volume de memória.

4.3 Requisitos de Segurança do Sistema criptográfico

4.3.1 Segurança do Ascon-AEAD128

Para o uso seguro do Ascon-AEAD128, existem requisitos operacionais especificados pelo NIST ([TURAN et al., 2025](#)). O primeiro requisito trata da geração da chave criptográfica, que deve seguir os padrões descritos no documento SP 800-133r2 [Barker, Roginsky e Davis \(2020\)](#), no qual é estabelecida a seguinte operação:

$$K = U \oplus V,$$

em que U é uma cadeia de bits extraída de um Gerador de Bits Aleatórios (RBG - *Random Bit Generator*) aprovado, e V consiste em uma sequência de bits gerada independentemente de U , podendo ser composta inteiramente por zeros.

As especificações para os geradores de bits aleatórios aprovados estão descritas no documento SP 800-90A [Barker e Kelsey \(2015\)](#). Neste projeto, será utilizado um Gerador Determinístico de Bits Aleatórios (DRBG - *Deterministic Random Bit Generator*) através do mecanismo CTR_DRBG. Essa escolha justifica-se pelo fato de que a entropia bruta extraída diretamente do hardware pode conter vícios físicos, tornando indispensável um estágio secundário de tratamento e condicionamento algorítmico ([TURAN et al., 2018](#)). Dessa forma, a implementação fará uso da biblioteca nativa *mbedTLS* do ESP32, que extrai a entropia verdadeira do periférico de hardware (TRNG) e a processa através do algoritmo CTR_DRBG, garantindo uma saída perfeitamente balanceada para a geração da chave.

Com base nesses elementos, os requisitos de segurança do Ascon-AEAD128 podem ser organizados em requisitos funcionais e não funcionais.

4.3.1.1 Requisitos funcionais

- RF1. A implementação da biblioteca *ascon-suite* deve ser validada previamente, conforme discutido na Seção 3.4.
- RF2. A geração da chave criptográfica deve seguir os padrões descritos no documento SP 800-133r2 [Barker, Roginsky e Davis \(2020\)](#), adotando a operação $K = U \oplus V$.
- RF3. O sistema deve utilizar um Gerador Determinístico de Bits Aleatórios (DRBG - *Deterministic Random Bit Generator*) através do mecanismo CTR_DRBG para a geração de chaves criptográficas.
- RF4. A implementação deve fazer uso da biblioteca nativa *mbedTLS* do ESP32 para extrair a entropia verdadeira do periférico de hardware (TRNG) e processá-la através do algoritmo CTR_DRBG.
- RF5. Caso a renovação da chave simétrica se faça necessária por eventuais comprometimentos ou políticas de segurança, o sistema deve adotar o método de encapsulamento de chaves (*key wrapping*), utilizando a chave secreta atual, desde que ainda válida e segura, para cifrar a nova chave gerada e permitir seu compartilhamento e atualização de forma segura entre o dispositivo e o nó de borda (*Edge*).

4.3.1.2 Requisitos não funcionais

- RNF1. A entropia bruta extraída diretamente do hardware não deve ser utilizada sem tratamento adicional, uma vez que pode conter vícios físicos (*bias*), tornando indispensável um estágio secundário de condicionamento algorítmico (TURAN et al., 2018).
- RNF2. A chave criptográfica deve ser mantida em sigilo absoluto, aliada à garantia de unicidade do *nonce* para cada operação de encriptação.
- RNF3. Para *tags* de autenticação cujo tamanho λ (em bits) satisfaça a condição $64 \leq \lambda \leq 128$, a quantidade de falhas de deciptação não deve exceder $2^{\lambda-32}$.
- RNF4. O volume máximo de dados processados durante as rotinas de encriptação e deciptação não deve ultrapassar 2^{54} bytes para uma mesma chave, devendo a chave ser obrigatoriamente renovada caso esse limite seja atingido.

Diante desses requisitos, alguns deles possuem impacto prático reduzido no contexto deste projeto. Como as redes LoRaWAN operam com taxas de transmissão extremamente baixas e pacotes pequenos, atingir o limite de 2^{54} bytes processados sob uma mesma chave é inviável durante a vida útil física do dispositivo. Da mesma forma, considerando o uso de uma *tag* de 128 bits, o limite tolerado antes da obrigatoriedade de rotação da chave é de 2^{96} falhas de deciptação. Alcançar esse volume de tentativas de ataques de injeção na rede é improvável, dadas as severas restrições de tempo de transmissão inerentes ao protocolo.

5 Desenvolvimento do Trabalho

5.1 Atividades Previstas

5.1.1 Implementação da infraestrutura

A implementação da infraestrutura corresponde à etapa inicial de desenvolvimento da solução proposta. Nessa fase, o objetivo é estabelecer a base de comunicação e persistência do sistema, permitindo integrar os nós de campo, o gateway e a camada de backend antes da incorporação dos mecanismos de segurança e análise.

Inicialmente, a implementação deverá concentrar-se na comunicação entre os dispositivos de campo e o gateway, utilizando placas com suporte integrado a LoRa. Os nós serão responsáveis pela coleta dos dados de aplicação e pela transmissão de mensagens curtas, enquanto o gateway atuará como ponto de recepção e transição entre a rede de campo e a infraestrutura de aplicação.

Após a recepção das mensagens, o gateway deverá organizar os dados recebidos, registrar metadados associados à comunicação e encaminhar essas informações ao backend por meio de conectividade Wi-Fi e requisições HTTP. No backend, será implementada uma API com mecanismo de autenticação, de modo a restringir o recebimento de dados a gateways autorizados.

Além do encaminhamento das mensagens, essa etapa também deverá contemplar a persistência dos dados de aplicação e dos metadados de comunicação em banco de dados. Esse armazenamento é necessário para o acompanhamento do funcionamento do sistema e para dar suporte às etapas posteriores do trabalho, especialmente aquelas relacionadas à segurança e à análise do comportamento da rede.

Por fim, a implementação da infraestrutura deverá prever um mecanismo básico de robustez para falhas temporárias de conectividade entre gateway e backend. Para isso, admite-se o uso de armazenamento local temporário no gateway, com reenvio posterior das mensagens quando a conexão for restabelecida. Com essa etapa, estabelece-se a base operacional mínima da solução, sobre a qual serão incorporadas, posteriormente, as camadas de criptografia leve e de aprendizado de máquina.

5.1.2 Segurança com Ascon

Para o desenvolvimento da camada de segurança com o padrão Ascon, o trabalho seguirá um roteiro de atividades práticas. O objetivo é organizar as etapas de implementação, validação e coleta de dados diretamente no hardware.

As atividades previstas para essa frente de trabalho são:

- **Integração da biblioteca:** Instalar e configurar a biblioteca *ascon-suite* no ambiente do ESP32, criando o código necessário para conectar a criptografia com o envio de pacotes LoRaWAN.
- **Validação Matemática (KAT):** Rodar os testes oficiais (*Known Answer Tests*) no computador para garantir que a biblioteca calcula as cifras corretamente. Depois, embarcar uma parte desses testes no próprio ESP32 para confirmar que não ocorreram erros durante a compilação para o microcontrolador.
- **Geração de Chaves:** Escrever o código para gerar as chaves criptográficas. Para isso, será usado o gerador de números aleatórios do hardware do ESP32 (TRNG) em conjunto com a função *CTR_DRBG* da biblioteca *mbedtls*, seguindo as recomendações oficiais do NIST.
- **Testes de Injeção de Falhas:** Criar um código de estresse que altere os pacotes de propósito (*bit-flipping*). A ideia é corromper o texto cifrado, o cabeçalho LoRaWAN e a *tag* de autenticação para provar que a função de descriptografia bloqueia a mensagem imediatamente.
- **Coleta de Métricas (Benchmarking):** Realizar as medições práticas de desempenho. Isso inclui contar os ciclos de *clock* para o tempo de execução, monitorar os picos de memória (Heap e Stack) e medir o consumo de energia com o osciloscópio e o resistor *shunt*.

5.1.3 Modelo de Machine Learning

O desenvolvimento da frente de aprendizado de máquina seguirá um conjunto de atividades voltadas à preparação dos dados, treinamento dos modelos e avaliação da capacidade de detecção de anomalias. O objetivo dessa etapa é construir uma base experimental consistente para comparar diferentes abordagens de regressão temporal aplicadas ao comportamento da rede LoRaWAN.

As atividades previstas para essa frente de trabalho são:

- **Preparação dos datasets:** Organizar os dados selecionados para a etapa de modelagem, com foco principal no dataset Tour Perret e uso complementar do dataset GUARD. Essa preparação incluirá harmonização de timestamps, tratamento de valores ausentes, extração dos atributos relevantes e organização das observações por dispositivo.
- **Definição e seleção de variáveis:** Avaliar os atributos disponíveis nos datasets e selecionar o subconjunto de variáveis mais adequado para a modelagem temporal

do comportamento normal da rede. Entre os candidatos principais estão RSSI, SNR, FCnt, *inter-arrival time* e *data rate*, considerando sua estabilidade temporal, disponibilidade e relevância para a identificação de desvios.

- **Divisão temporal dos dados:** Separar os dados em conjuntos de treinamento (60%), validação (20%) e teste (20%) por meio de uma divisão cronológica, preservando a ordem temporal das observações e evitando vazamento de informação futura para o treinamento dos modelos.
- **Exploração e seleção de arquiteturas:** Implementar os modelos ARIMA, RNN e LSTM definidos na etapa metodológica do trabalho. Para cada modelo, será realizada uma exploração de hiperparâmetros e configurações arquiteturais com o objetivo de identificar combinações adequadas ao problema. A seleção de configurações será orientada por critérios de desempenho preditivo e simplicidade estrutural, priorizando modelos com menor complexidade quando o ganho de performance for marginal.
- **Treinamento exclusivo em dados normais:** Todos os modelos serão treinados e validados exclusivamente com dados representativos da operação normal da rede (dataset Tour Perret), garantindo que o comportamento esperado seja aprendido sem contaminação por anomalias ou ataques.
- **Avaliação da qualidade preditiva:** Comparar os modelos com base em métricas de erro de regressão (MAE e RMSE) no conjunto de teste, com o objetivo de verificar qual abordagem representa com maior fidelidade o comportamento esperado das variáveis monitoradas.
- **Calibração de limiar de detecção:** Com base nos resíduos do conjunto de validação, será definido um critério de limiarização (baseado em desvio padrão ou percentis) para classificar observações como normais ou potencialmente anômalas.
- **Avaliação da capacidade de detecção:** Analisar a performance de detecção dos modelos no conjunto de teste e também na aplicação aos dados do dataset GUARD, com o objetivo de verificar se os modelos conseguem identificar anomalias quando confrontados com padrões diferentes ou com ataques conhecidos. Será utilizada a métrica de teste de generalização para avaliar robustez.
- **Análise de viabilidade para execução em borda:** Após a comparação entre os modelos, será realizada uma análise complementar da complexidade das soluções obtidas, considerando número de parâmetros, custo computacional e compatibilidade com uma eventual etapa futura de adaptação para TinyML.

5.1.4 Integração progressiva da solução

A implementação proposta neste trabalho foi organizada de forma progressiva, de modo que cada etapa estabeleça a base necessária para a etapa seguinte. Inicialmente, a prioridade está na consolidação da infraestrutura de comunicação e persistência, garantindo que os dados produzidos em campo possam ser recebidos, encaminhados, autenticados e armazenados de forma consistente.

Sobre essa base, será incorporada a camada de segurança com o uso do padrão Ascon, com foco na proteção do payload gerado pelos nós de campo. Essa etapa permitirá avaliar a inserção dos mecanismos criptográficos sem alterar a organização principal da infraestrutura já estabelecida, preservando a separação entre comunicação, autenticação e persistência.

Em seguida, a base de dados formada pela operação da infraestrutura será utilizada na etapa de análise por aprendizado de máquina. Nessa fase, tanto os dados de aplicação quanto os metadados de comunicação poderão ser explorados com o objetivo de apoiar a análise do comportamento da rede e a identificação de padrões associados a anomalias.

Dessa forma, a proposta não trata infraestrutura, segurança e análise como frentes independentes, mas como partes complementares de uma mesma solução. A integração progressiva dessas camadas permite desenvolver o sistema com maior clareza, reduz a complexidade da implementação inicial e favorece a avaliação da arquitetura como um todo.

6 Considerações Preliminares

Até a presente etapa de desenvolvimento do trabalho, os esforços foram concentrados na consolidação da fundamentação teórica, na revisão bibliográfica e na especificação da arquitetura do sistema. O trabalho estabeleceu a base conceitual necessária para a integração segura de dispositivos IoT em redes LoRaWAN, definindo os papéis do nó sensor, do *gateway* e da infraestrutura de *backend*.

Durante a fase de especificação, foi possível justificar tecnicamente as escolhas das tecnologias que comporão o protótipo. O padrão Ascon-AEAD128 foi selecionado para garantir a confidencialidade e integridade dos dados em trânsito com baixo custo computacional, sobrepondo-se a alternativas legadas devido à sua adequação a ambientes restritos. Em paralelo, a estratégia de aprendizado de máquina foi definida, optando-se por modelos de regressão temporal não supervisionados (utilizando os *datasets* Tour Perret e GUARD) como mecanismo de detecção de anomalias operacionais.

6.1 Perspectivas de Continuidade

O presente trabalho estabelece uma arquitetura de segurança integrada para redes LoRaWAN, combinando criptografia leve por meio do padrão Ascon e detecção de anomalias via modelos de regressão temporal. Embora os objetivos definidos para este projeto sejam tratados de forma completa nas etapas descritas nos capítulos anteriores, a natureza do problema abre caminhos relevantes para investigações futuras.

A perspectiva de maior interesse imediato diz respeito à portabilidade da fase de inferência dos modelos treinados para o próprio gateway da rede. Neste trabalho, o treinamento e a avaliação dos modelos são realizados em ambiente computacional convencional, decisão justificada pelas restrições de hardware incompatíveis com o processo de treinamento. Contudo, uma vez obtido um modelo com desempenho satisfatório, torna-se pertinente investigar se sua fase de inferência pode ser executada diretamente no gateway, eliminando a necessidade de transmissão dos dados brutos para uma camada externa de processamento.

Essa investigação demandaria a análise da complexidade do modelo resultante — em termos de número de parâmetros, tamanho serializado e tempo de inferência — em relação às restrições de memória RAM e armazenamento em *flash* do hardware adotado. Ferramentas como o TensorFlow Lite for Microcontrollers e a biblioteca *emlearn* permitem a conversão de modelos treinados em Python para código C otimizado, compatível com microcontroladores como o ESP32 (RAY, 2022). Técnicas de compressão como quantização

de pesos e poda de conexões poderiam ser aplicadas para reduzir o custo computacional sem perda significativa de desempenho (JACOB et al., 2018).

Uma segunda perspectiva relevante envolve a extensão da abordagem de detecção para incorporar dados rotulados de ataque, como os disponíveis no *dataset* GUARD. Com resultados de regressão consolidados sobre o Tour Perret, seria possível investigar em que medida os resíduos produzidos pelos modelos se comportam de forma diferenciada diante de *frames* reconhecidamente manipulados, abrindo caminho para uma avaliação mais rigorosa da capacidade de detecção em cenários com ataques conhecidos.

Por fim, a arquitetura proposta poderia ser estendida para contemplar mecanismos de atualização contínua do modelo, permitindo que o sistema se adapte a mudanças graduais no comportamento da rede — como variações sazonais de sinal ou substituição de dispositivos — sem comprometer a sensibilidade à detecção de anomalias. Essa capacidade de adaptação é especialmente relevante em implantações de longa duração, características de aplicações industriais e de cidades inteligentes.

6.1.1 Segurança Física do ESP32

Embora os resultados deste trabalho procurem demonstrar que o Ascon-AEAD128 fornece segurança robusta e viabilidade de processamento para os dados em trânsito pela rede LoRaWAN, é fundamental ressaltar que a integridade do sistema como um todo depende do sigilo da chave criptográfica armazenada no nó sensor. Como dispositivos de borda (*Edge*) operam em ambientes fisicamente hostis e não supervisionados, eles se mantêm como alvos potenciais para ataques de extração de memória (*Flash Dumping*).

Neste contexto, convém destacar que o escopo analítico desta pesquisa concentrou-se na eficiência das primitivas criptográficas. Contudo, para a transição desse modelo para uma implantação real, a segurança em nível de hardware atua como uma camada complementar e indispensável.

Como recomendação para mitigar essas vulnerabilidades físicas em desenvolvimentos futuros ou aplicações comerciais baseadas nesta arquitetura, sugere-se a adoção conjunta dos mecanismos nativos de segurança da família ESP32 (Espressif Systems, 2024) (KÖSEMEN; DALKILIÇ, 2021). A proteção da chave simétrica do Ascon pode ser efetivada utilizando o recurso de *Flash Encryption* sobre a partição não-volátil (NVS), com a chave mestre gravada nos *eFuses* do microcontrolador (OTP) e protegida contra leitura externa (*Read Protect*). Somado a isso, a implementação da Inicialização Segura (*Secure Boot*), verificando a assinatura digital do *firmware* a cada inicialização, previne ataques de regravação maliciosa. Dessa forma, garante-se que a eficiência criptográfica comprovada do Ascon opere sobre uma fundação de hardware onde os dados em repouso e o código de execução estejam devidamente protegidos contra o acesso físico não autorizado.

Referências

- AL-FUQAHA, A. et al. Internet of things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, v. 17, n. 4, p. 2347–2376, Fourthquarter 2015. Disponível em: <https://doi.org/10.1109/COMST.2015.2444095>. Acesso em: 9 abr. 2026. Citado na página 21.
- BALDI, M. et al. *SPECK: Signatures from Permutation Equivalence of Codes and Kernels*. 2025. Cryptology ePrint Archive, Paper 2025/923. Disponível em: <https://eprint.iacr.org/2025/923>. Acesso em: 15 mar. 2026. Citado na página 36.
- BARKER, E.; KELSEY, J. *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*. Gaithersburg, MD, 2015. (NIST Special Publication 800-90A Revision 1). Disponível em: <https://doi.org/10.6028/NIST.SP.800-90Ar1>. Acesso em: 19 mar. 2026. Citado na página 63.
- BARKER, E.; ROGINSKY, A.; DAVIS, R. *Recommendation for Cryptographic Key Generation*. Gaithersburg, MD, 2020. (NIST Special Publication 800-133 Revision 2). Disponível em: <https://doi.org/10.6028/NIST.SP.800-133r2>. Acesso em: 19 mar. 2026. Citado 2 vezes nas páginas 62 e 63.
- BERGMEIR, C.; BENÍTEZ, J. M. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, v. 191, p. 192–213, 2012. Citado na página 51.
- BERNSTEIN, D. J. *ChaCha, a variant of Salsa20*. 2008. Website pessoal. Disponível em: <https://cr.yp.to/chacha.html>. Acesso em: 15 mar. 2026. Citado na página 36.
- BOX, G. E. P. et al. *Time Series Analysis: Forecasting and Control*. 5. ed. Hoboken: Wiley, 2015. Citado 2 vezes nas páginas 39 e 51.
- CAESAR Committee. *Submissions to the CAESAR Competition*. 2014. CAESAR: Competition for Authenticated Encryption. Disponível em: <https://competitions.cr.yp.to/caesar-submissions.html>. Acesso em: 12 mar. 2026. Citado na página 22.
- CampusIoT. *CampusIoT Platform*. 2023. Project website. Disponível em: <https://www.campusiot.imag.fr/>. Acesso em: 15 mar. 2026. Citado na página 46.
- CampusIoT. *The Tour Perret LoRaWAN Frames Dataset*. 2023. Dataset repository. Disponível em: <https://github.com/campusiot/tour-perret-lorawan-frames-dataset>. Acesso em: 15 mar. 2026. Citado na página 46.
- CHALAPATHY, R.; CHAWLA, S. Deep learning for anomaly detection: A survey. *arXiv preprint*, arXiv:1901.03407, 2019. Disponível em: <https://arxiv.org/abs/1901.03407>. Citado 3 vezes nas páginas 22, 38 e 52.
- CHEN, L.; SCHOLL, M. *The Cornerstone of Cybersecurity – Cryptographic Standards and a 50-Year Evolution*. 2022. NIST Cybersecurity Insights Blog. Disponível em: <https://www.nist.gov/blogs/cybersecurity-insights/cornerstone-cybersecurity-cryptographic-standards-and-50-year-evolution>. Acesso em: 15 mar. 2026. Citado na página 34.

DOBRAUNIG, C. et al. *Ascon: Lightweight Cryptography for Constrained Devices*. 2023. Website oficial. Disponível em: <<https://ascon.isec.tugraz.at/>>. Acesso em: 12 mar. 2026. Citado na página 22.

DOBRAUNIG, C. et al. *Reference implementations of Ascon (v1.2)*. 2023. GitHub Repository. Disponível em: <<https://github.com/ascon/ascon-c>>. Acesso em: 7 abr. 2026. Citado na página 55.

Espressif Systems. *Flash Encryption: ESP32 Security Features*. 2024. ESP-IDF Programming Guide. Disponível em: <<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/security/flash-encryption.html>>. Acesso em: 20 mar. 2026. Citado na página 70.

FARHAD, M. et al. Machine learning techniques for lorawan networks: A survey. *IEEE Access*, v. 11, p. 1–20, 2023. Citado na página 37.

GUARD Project. *LoRaWAN Network Attacks Dataset*. 2023. Dataset repository. Disponível em: <<https://github.com/guard-project/lorawan-network-attacks>>. Acesso em: 15 mar. 2026. Citado na página 48.

HAYASHI, V. T.; RUGGIERO, W. V. Hands-free authentication for virtual assistants with trusted iot device and machine learning. *Sensors*, MDPI, v. 22, n. 4, p. 1325, 2022. Disponível em: <<https://doi.org/10.3390/s22041325>>. Citado na página 22.

HESSEL, F.; ALMON, L.; HOLLICK, M. Lorawan security: An evolvable survey on vulnerabilities, attacks and their systematic mitigation. *ACM Transactions on Sensor Networks*, v. 18, n. 4, p. Article 70, Mar. 2023. Disponível em: <<https://doi.org/10.1145/3561973>>. Acesso em: 15 mar. 2026. Citado 11 vezes nas páginas 21, 22, 25, 26, 27, 28, 29, 30, 31, 50 e 52.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997. Citado na página 40.

Hubble. *How to Profile ESP32 Power Consumption Accurately*. 2024. Hubble Community Guides. Disponível em: <<https://hubble.com/community/guides/how-to-profile-esp32-power-consumption-accurately/>>. Acesso em: 7 abr. 2026. Citado 2 vezes nas páginas 57 e 58.

HYNDMAN, R. J.; ATHANASOPOULOS, G. *Forecasting: Principles and Practice*. 3. ed. Melbourne: OTexts, 2021. Disponível em: <<https://otexts.com/fpp3/>>. Citado 2 vezes nas páginas 51 e 52.

JACOB, B. et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [s.n.], 2018. Disponível em: <<https://arxiv.org/abs/1712.05877>>. Citado 2 vezes nas páginas 41 e 70.

KÖSEMEN, C.; DALKILIÇ, G. Tamper resistance functions on internet of things devices. In: *2021 Innovations in Intelligent Systems and Applications Conference (ASYU)*. [s.n.], 2021. p. 1–5. Disponível em: <<https://doi.org/10.1109/ASYU52992.2021.9598995>>. Acesso em: 20 mar. 2026. Citado na página 70.

- MCCARTHY, K. *NSA design for IoT encryption rejected over spying fears*. 2018. The Register. Disponível em: <https://www.theregister.com/2018/04/25/nsa_iot_encryption/>. Acesso em: 7 abr. 2026. Citado na página 36.
- MCKAY, K. A. et al. *Report on Lightweight Cryptography*. [S.l.], 2017. Disponível em: <<https://doi.org/10.6028/NIST.IR.8114>>. Acesso em: 15 mar. 2026. Citado 2 vezes nas páginas 21 e 34.
- NIST. *Lightweight Cryptography*. 2017. Computer Security Resource Center (CSRC). Disponível em: <<https://csrc.nist.gov/projects/lightweight-cryptography>>. Acesso em: 15 mar. 2026. Citado na página 35.
- NIST. *Lightweight Cryptography: Finalists*. 2021. Computer Security Resource Center (CSRC). Disponível em: <<https://csrc.nist.gov/projects/lightweight-cryptography/finalists>>. Acesso em: 12 mar. 2026. Citado 2 vezes nas páginas 22 e 37.
- NIST. *Cybersecurity Framework Version 2.0*. 2024. <<https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf>>. Acesso em: 12 abr. 2024. Citado na página 25.
- NIST. *NIST Finalizes Lightweight Cryptography Standard to Protect Small Devices*. 2025. NIST News. Disponível em: <<https://www.nist.gov/news-events/news/2025/08/nist-finalizes-lightweight-cryptography-standard-protect-small-devices>>. Acesso em: 15 mar. 2026. Citado 2 vezes nas páginas 35 e 36.
- PANDEY, A.; DIWAN, P.; DHAR, T. A scalable cryptographic framework for healthcare iot using optimized speck, tinyjambu, and ascon. *International Journal of Innovative Research in Computer and Communication Engineering*, 2025. Citado na página 23.
- RAY, P. P. A review on TinyML: State-of-the-art and prospects. *Journal of King Saud University – Computer and Information Sciences*, v. 34, n. 4, p. 1595–1623, 2022. Citado 2 vezes nas páginas 41 e 69.
- RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, p. 533–536, 1986. Citado na página 40.
- TURAN, M. S. et al. *Recommendation for the Entropy Sources Resulting in Random Bit Generation*. Gaithersburg, MD, 2018. (NIST Special Publication 800-90B). Disponível em: <<https://doi.org/10.6028/NIST.SP.800-90B>>. Acesso em: 19 mar. 2026. Citado 2 vezes nas páginas 63 e 64.
- TURAN, M. S. et al. *Status Report on the Final Round of the NIST Lightweight Cryptography Standardization Process*. [S.l.], 2023. Disponível em: <<https://doi.org/10.6028/NIST.IR.8454>>. Acesso em: 15 mar. 2026. Citado 2 vezes nas páginas 37 e 54.
- TURAN, M. S. et al. *Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions*. Gaithersburg, MD, 2025. (NIST Special Publication 800-232). Disponível em: <<https://doi.org/10.6028/NIST.SP.800-232>>. Acesso em: 12 mar. 2026. Citado 2 vezes nas páginas 21 e 62.
- VANGELISTA, L. et al. Data-driven optimization in lorawan: Challenges and opportunities. *Sensors*, v. 23, n. 1, p. 1–18, 2023. Citado na página 37.

WEATHERLEY, R. *Ascon Suite: C implementations of the Ascon lightweight cryptography family*. 2023. GitHub Repository. Disponível em: <<https://github.com/rweather/ascon-suite>>. Acesso em: 15 mar. 2026. Citado na página 53.